

The Syndrome Decoding in the Head (SD-in-the-Head) Signature Scheme

Algorithm Specifications and Supporting Documentation – Version 2.0

Carlos Aguilar Melchor Slim Bettaieb Loïc Bidoux
Thibauld Feneuil Philippe Gaborit Nicolas Gama
Shay Gueron James Howe Andreas Hülsing David Joseph
Antoine Joux Mukul Kulkarni Edoardo Persichetti
Tovohery H. Randrianarisoa Matthieu Rivain Dongze Yue

February 5, 2025

CISPA
CryptoExperts
Eindhoven University of Technology
Florida Atlantic University
Meta
SandboxAQ
Sapienza University
Technology Innovation Institute
University of Haifa
University of Limoges

Contents

1	Introduction	2
2	High-level description	3
2.1	VOLEitH in the PIOP formalism	3
2.2	Overview of SD-in-the-Head-2	7
3	Detailed algorithmic description	11
3.1	Main algorithms	11
3.1.1	Key generation	11
3.1.2	Signing	12
3.1.3	Verification	13
3.2	Subroutines	15
3.2.1	Symmetric cryptography primitives	15
3.2.2	Pseudo-randomness generation	15
3.2.3	Hashing and commitments	18
3.2.4	All-but-one vector commitments	19
3.2.5	Batch line commitment	22
3.2.6	PIOP protocol	28
4	Parameters and performances	35
4.1	Selection of parameters	35
4.2	Keys and signature sizes	36
4.3	Selected parameters	37
4.4	Benchmarks	37
5	Security	39
5.1	SD to RSD security reduction	39
5.2	Attacks against the SD problem	40
5.3	Unforgeability	41
6	Variants	42
6.1	The $\mathbb{F}_{256}$ variant	42
6.2	The TCitH variant	43
7	Advantages and limitations	45
7.1	Advantages of SD-in-the-Head	45
7.2	Limitations of SD-in-the-Head	45

Changelog

2025-02-05: Version 1.1 → Version 2.0

We have updated the SDitH signature scheme to align it with the current literature. The first version of the scheme utilized the **MPCitH framework** using linear multiparty computation, optimized through **hypercube** [AGH⁺23] and **threshold approaches** [FR23b]. In the second version, we have adopted the **VOLEitH framework** [BBD⁺23], which allows us to reduce the signature size by more than half.

Additionally, we have modified the syndrome decoding field. Previously, we focused on $\mathbb{F}_{256}$ and $\mathbb{F}_{251}$; now, we are using the binary field $\mathbb{F}_2$, which offers a more conservative security assumption.

Finally, we have redesigned the seed trees to incorporate AES-128 and Rijndael-256-256 as symmetric primitives, moving away from Keccak-based hashes to enhance the speed of the scheme.

1 Introduction

This specification presents the Syndrome-Decoding-in-the-Head (SD-in-the-Head) digital signature scheme. The scheme is based on the hardness of the syndrome decoding (SD) problem for random linear codes on a finite field. It consists in a zero-knowledge proof of knowledge of a low-weight vector x solution of a syndrome decoding instance $y = Hx$, which is made non-interactive using the Fiat-Shamir transform. This zero-knowledge proof relies on the principle of “multiparty computation in the head” (MPCitH) originally introduced in [IKO⁺07] and notably used by the Picnic signature scheme [ZCD⁺20], candidate to the previous NIST call for post-quantum algorithms. The first version of SD-in-the-Head was based on the initial scheme published in [FJR22] with further improvements from [AGH⁺22; FR22]. This second version relies on a different proof system. This proof system is based on the VOLE-in-the-Head framework [BBD⁺23] and its efficient application to the SD problem [OTX24; BBG⁺24].

For most applications of signatures, specially the ones that require certificates such as TLS, the “public key + signature size” is thus a critical metric. The SD-in-the-Head signature scheme presented in this specification achieves 3.7 KB for this metric in Category I, which is similar to ML-DSA and less than halved compared to SLH-DSA (7.8 KB).

Organization of this specification

Section 2 gives a high-level description of the SD-in-the-Head-2 scheme. **Section 3** provides a detailed description of the key generation, signature and verification algorithms. This description intends to allow a non-ambiguous implementation of the scheme. The selection of the parameters is explained in **Section 4** which also exhibits our proposed instances for the three considered security levels. **Section 4.4** provides performance figures for our different instances. The security of the SD-in-the-Head signature scheme is analyzed in **Section 5** while **Section 5.2** further evaluates the complexity of known attacks. We finally list some advantages and limitations of the scheme in **Section 7**.

We welcome enquiries, comments, and corrections at

consortium@sdith.org

Implementations and material related to the SD-in-the-Head signature scheme will be uploaded and maintained on:

<https://github.com/sdith>

2 High-level description

The second version of the SD-in-the-Head signature scheme, namely SD-in-the-Head-2, relies on the VOLE-in-the-Head framework [BBD⁺23] to build a 7-round zero-knowledge proof of knowledge for the syndrome decoding problem, which is then transformed into a signature scheme using the Fiat-Shamir heuristic [FS87]. The modeling used to handle the syndrome decoding problem in the VOLEitH framework has been proposed independently into the two articles [OTX24] and [BBG⁺24].

The following sections present the VOLE-in-the-Head framework using the PIOP formalism and describe how SD-in-the-Head-2 is built from this framework and the modeling of [OTX24; BBG⁺24].

2.1 VOLEitH in the PIOP formalism

The MPCitH paradigm [IKO⁺07] is a versatile method introduced in 2007 to build zero-knowledge proof systems using techniques from secure multi-party computation (MPC). This paradigm has been drastically improved in recent years and is particularly efficient to build zero-knowledge proofs for small circuits such as those involved in (post-quantum) signature schemes. The more recent MPCitH-based frameworks are the VOLE-in-the-Head (VOLEitH) framework from [BBD⁺23] and the Threshold-Computation-in-the-Head (TCitH) framework from [FR23b; FR23a].

In this subsection, we will describe the general proof system which SD-in-the-Head-2 relies on. In what follows, we present this proof system using the formalism of the Polynomial Interactive Oracle Proofs (PIOP), as presented in [Fen24]. Indeed, while the TCitH and VOLEitH frameworks were originally introduced using sharing-based and VOLE-based formalisms, respectively, we present them within the PIOP formalism, which provides a unified and comprehensive ground for these techniques.¹

Let us assume that we want to build an interactive zero-knowledge proof with a prover convincing a verifier that they know a witness $w \in \mathbb{F}_2^k$ which satisfies some public polynomial relations:

$$f_j(w) = 0, \forall 1 \leq j \leq m$$

where $f_1, \dots, f_m$ are polynomials over $\mathbb{F}_2$ of total degree at most d . Let us consider a public subset $S \subset \mathbb{F}_{2^\lambda}$. The proof system we consider is the following:

1. For $1 \leq j \leq n$, the prover samples a random degree-1 polynomials P_j such that $P_j(0) = w_j$. They also sample a random degree- $(d-1)$ polynomial $P_0 \in \mathbb{F}_{2^\lambda}[X]$. They commit to those polynomials.
2. The verifier chooses random coefficients $\gamma_1, \dots, \gamma_m$ from $\mathbb{F}_{2^\lambda}$ and sends them to the prover. The latter then reveals the degree- $(d-1)$ polynomial $Q(X)$ defined such that

$$Q(X) \cdot X = P_0(X) \cdot X + \sum_{j=1}^m \gamma_j \cdot f_j(P_1(X), \dots, P_n(X)). \quad (1)$$

¹In the TCitH framework, instead of performing operations over Shamir's secret sharings, we can directly work over their underlying polynomials. In the VOLEitH framework, instead of performing operations over VOLE gadgets, we can directly work over their underlying degree-1 polynomials.

3. The verifier samples a random evaluation point Δ from the public set $S \subset \mathbb{F}_{2^\lambda}$ and sends it to the prover. The latter then reveals the evaluations $v_i := P_i(\Delta)$, together with a proof π that the evaluations are consistent with the commitment.
4. The verifier checks that the revealed evaluations are consistent with the commitment using π and checks that we have

$$Q(\Delta) \cdot \Delta = v_0 \cdot \Delta + \sum_{j=1}^m \gamma_j \cdot f_j(v_1, \dots, v_n) . \quad (2)$$

The above protocol assumes that the prover has a way to commit polynomials and to provably open some evaluations later (while keeping hidden the other evaluations).

Security analysis. We can observe that the coefficient in front of the degree-0 monomial (*i.e.* the constant term) in the right term of Equation (1) is

$$\sum_{j=1}^m \gamma_j \cdot f_j(w_1, \dots, w_n) , \quad \text{ } \quad (3)$$

so the degree- $(d-1)$ polynomial Q is well-defined because this quantity is zero when the witness w is valid. Let us assume that the prover is malicious, meaning that they do not know a valid witness. It implies that there exists j^* such that $f_{j^*}(w) \neq 0$. In that case, the probability that there exists some Q such that Equation (1) holds is at most $1/2^\lambda$ over the randomness of $\gamma_1, \dots, \gamma_m$, because the coefficient (3) is zero only with probability $1/2^\lambda$. If Equation (1) does not hold, the probability that the check in Equation (2) passes is at most $d/|S|$, since the degree- d polynomial relation

$$Q(X) - \left(P_0 + \sum_{j=1}^m \gamma_j \cdot f_j^{[h]}(X, P_1(X), \dots, P_m(X)) \right) \neq 0$$

would have at most d roots (and so the random challenge Δ should be among those roots). So, the proof system is sound, with a soundness error of $\frac{1}{2^\lambda} + \left(1 - \frac{1}{2^\lambda}\right) \cdot \frac{d}{|S|}$. Moreover, assuming that the commitment scheme is hiding, we can observe that the interactive proof is zero-knowledge since

- revealing $Q(X)$ leaks no information about the secret thanks to the random polynomial P_0 , and
- revealing one evaluation of the polynomials $P_1, \dots, P_n$ leaks no information about the leading term thanks to the randomness used to build those polynomials.

In what follows, we describe how to commit polynomials such that we can later open some evaluations.

The TCitH-GGM approach. Thanks to ideas from [ISN89; CDI05], the TCitH framework [FR23a] shows that we can commit $\tilde{n}$ random polynomials using seed trees in such a way that the committer can later open one evaluation among a set of N while keeping the others hidden. Here is the commitment process for degree-1 polynomials:

1. One uses an all-but-one vector commitment (AVC) to sample and commit N seeds $\text{seed}_1, \dots, \text{seed}_N$.
2. One expands each seed_i as $w_{\text{rnd},i} := \text{PRG}(\text{seed}_i) \in \mathbb{F}_q^{\bar{n}}$ for $i \in \{1, \dots, N\}$, where PRG is a pseudorandom generator.
3. One computes

$$w_{\text{acc}} \leftarrow \sum_{i=1}^N w_{\text{rnd},i} \in \mathbb{F}_2^{\bar{n}}$$

$$w_{\text{base}} \leftarrow - \sum_{i=1}^N \phi(i) \cdot w_{\text{rnd},i} \in \mathbb{F}_{2^\kappa}^{\bar{n}}$$

where $\phi : \{1, \dots, N\} \rightarrow \mathbb{F}_{2^\kappa}$ is a public one-to-one function.

4. One defines P_j as

$$P_j(X) = (w_{\text{acc}})_j \cdot X + (w_{\text{base}})_j$$

for all j .

This commitment procedure has the main advantage to enable the prover to reveal one evaluation $\{P_j(\phi(i^*))\}_j$ for $i^* \in [1 : N]$ while keeping *secret* the coefficients w_{acc} and w_{base} : they just need to reveal all the $\{\text{seed}_i\}_i$ except seed_{i^*} (by opening the AVC scheme) and the verifier will be able to compute $P_j(\phi(i^*))$ as

$$\sum_{i=1, i \neq i^*}^N (\phi(i^*) - \phi(i)) \cdot (w_{\text{rnd},i})_j \quad \text{with} \quad w_{\text{rnd},i} := \text{PRG}(\text{seed}_i).$$

Indeed, we have that

$$\begin{aligned} \sum_{i=1, i \neq i^*}^N (\phi(i^*) - \phi(i)) \cdot (w_{\text{rnd},i})_j &= \phi(i^*) \cdot \sum_{i=1}^N (w_{\text{rnd},i})_j - \sum_{i=1}^N \phi(i) \cdot (w_{\text{rnd},i})_j \\ &= \phi(i^*) \cdot (w_{\text{acc}})_j + (w_{\text{base}})_j = P_j(\phi(i^*)). \end{aligned}$$

We can use this commitment procedure to commit to the polynomials in Step 1 of the proof system. We just need to rely on auxiliary values to enforce some coefficients of P_j . Because the public set would be of size N ($S := \{\phi(1), \dots, \phi(N)\}$), the resulting 5-round zero-knowledge proof system has a soundness error of

$$\frac{1}{2^\lambda} + \left(1 - \frac{1}{2^\lambda}\right) \cdot \frac{d}{N},$$

and one needs to rely on protocol repetitions to achieve the desired security. Indeed, the computational complexity is linear in N and so we can not take N exponentially large. To have a λ -bit security we need to repeat the protocol τ times in parallel, such that $(d/N)^\tau \leq 2^{-\lambda}$.

The VOLEitH approach. In the VOLEitH framework, the commitment scheme enables the opening of an evaluation from an exponentially-large set S , thus avoiding parallel repetitions of the PIOP. As the TCitH framework, the VOLEitH approach starts by committing τ sets of polynomials $\{P_i^{(1)}\}_i, \dots, \{P_i^{(\tau)}\}_i$ in parallel (exactly using the same commitment procedure). However, instead of considering those sets of polynomials individually as in the TCitH framework, the VOLEitH approach consists in “merging them” into a polynomial over an extension field of size greater than N^τ .

This merge works as follows. Consider τ polynomials $P^{(1)}, \dots, P^{(\tau)}$ encoding a witness coefficient in their leading terms: $P^{(e)} = w \cdot X + w_{\text{base}}^{(e)}$, where $w \in \mathbb{F}_2$ and $w_{\text{base}}^{(e)} \in \mathbb{F}_{2^\kappa}$ for every e . These can be merged into the polynomial:

$$P(X) = w \cdot X + \underbrace{\psi(w_{\text{base}}^{(1)}, \dots, w_{\text{base}}^{(\tau)})}_{\in \mathbb{F}_{2^\lambda}}$$

where $\tau \cdot \kappa \leq \lambda$ and ψ is an $\mathbb{F}_2$ -morphism from $(\mathbb{F}_{2^\kappa})^\tau$ to $\mathbb{F}_{2^\lambda}$. The key observation is that we can open an evaluation $P(X)$ into any point of

$$E := \{\psi(v_1, \dots, v_\tau) \mid (v_1, \dots, v_\tau) \in \{\phi(1), \dots, \phi(N)\}^\tau\} \subset \mathbb{F}_{2^\lambda}$$

by opening evaluations of $P^{(1)}, \dots, P^{(\tau)}$ on the small domain. Specifically, we compute $P(\Delta)$ where $\Delta = \psi(\phi(i^{(1)}), \dots, \phi(i^{(\tau)}))$ for some $(i^{(1)}, \dots, i^{(\tau)}) \in \{1, \dots, N\}^\tau$ as

$$\psi(P^{(1)}(\phi(i^{(1)})), \dots, P^{(\tau)}(\phi(i^{(\tau)}))) .$$

Indeed, we have

$$\begin{aligned} \psi(P^{(1)}(\phi(i^{(1)})), \dots, P^{(\tau)}(\phi(i^{(\tau)}))) &= \psi(w \cdot \phi(i^{(1)}) + w_{\text{base}}^{(1)}, \dots, w \cdot \phi(i^{(\tau)}) + w_{\text{base}}^{(\tau)}) \\ &= w \cdot \psi(\phi(i^{(1)}), \dots, \phi(i^{(\tau)})) + \psi(w_{\text{base}}^{(1)}, \dots, w_{\text{base}}^{(\tau)}) \\ &= w \cdot \Delta + \psi(w_{\text{base}}^{(1)}, \dots, w_{\text{base}}^{(\tau)}) = P(\Delta) . \end{aligned}$$

So, instead of running the proof system τ times in parallel over the polynomials $P^{(1)}, \dots, P^{(\tau)}$, one runs it once over the polynomial P . Since the evaluation set S is now of size N^τ , the resulting soundness error is thus

$$\frac{1}{2^\lambda} + \left(1 - \frac{1}{2^\lambda}\right) \cdot \frac{d}{N^\tau} .$$

Let us remark that the merge requires the polynomials $P^{(1)}, \dots, P^{(\tau)}$ to have the same leading term. Therefore, the prover should convince the verifier that those τ polynomials have the same leading term. Let us assume that we work with vector polynomials, *i.e.* $P^{(e)}$ is of the form $P^{(e)} = w \cdot X + w_{\text{base}}^{(e)}$ for some $w \in \mathbb{F}_2^{\bar{n}}$ and $w_{\text{base}}^{(e)} \in \mathbb{F}_{2^\lambda}^{\bar{n}}$. To convince the verifier, the prover will run the following consistency check. After committing to the τ polynomials, the prover gets a random matrix $M \in \mathbb{F}_2^{(\lambda+B) \times \bar{n}}$ from the verifier and then reveals the polynomial

$$R^{(e)}(X) \leftarrow M \cdot P^{(e)}(X)$$

for all $e \in \{1, \dots, \tau\}$. The verifier can then check that the leading terms of $R^{(1)}(X), \dots, R^{(\tau)}(X)$ are indeed the same, and, after the opening of the evaluations, they can check that

$$R^{(e)}(\phi(i^{(e)})) = M \cdot P^{(e)}(\phi(i^{(e)}))$$

for all e .

The probability that there exists e_1 and e_2 such that the leading terms of $R^{(e_1)}(X)$ and $R^{(e_2)}(X)$ are the same, while those of $P^{(e_1)}(X)$ and $P^{(e_2)}(X)$ are different is at most $\binom{\tau}{2} \cdot \varepsilon_{\text{check}}$ where $\varepsilon_{\text{check}} := \max_{v \neq u} \Pr_{\mathbf{M} \leftarrow \mathcal{S}}[\mathbf{M}v = \mathbf{M}u]$. The parameter B is chosen such that $\binom{\tau}{2} \cdot \varepsilon_{\text{check}} \leq 2^{-\lambda}$. Let us note that this check leaks information about the leading term w . To prevent leakage about secret values, the $\lambda + B$ last coefficients of w can be chosen at random and $\mathbf{M}$ can be defined in the form $\mathbf{M} = [\mathbf{M}' \mid \mathbf{I}_{\lambda+B}]$. Because of this consistency check which relies on a random challenge from the verifier, the **resulting zero-knowledge protocol has 7 rounds.**

Witness as constant term. In the proof system described at the beginning of this section, we use some polynomials $P_1, \dots, P_n$ such that $P_j(0) = w_j$ for all j , meaning the witness is encoded as the *constant term* of those polynomials. On the other hand, the VOLEitH commitment procedure merges those polynomials assuming that the witness is encoded as their *leading term*. In its original description, the TCitH framework relies on the former encoding (as for original Shamir's secret sharing) but it can easily support the latter encoding. On the other hand, the VOLEitH approach is constrained to use the leading-term encoding because ψ is $\mathbb{F}_2$ -linear and so the merging strategy requires that the leading term lives in $\mathbb{F}_2$ (and not in $\mathbb{F}_{2^\lambda}$).

One advantage of the constant-term encoding is to be less expensive in terms of field multiplications. Because of the relative heaviness of the considered modelling for SD-in-the-Head-2 we propose the following tweak to support VOLEitH merging procedure while still encoding the witness in the constant term. We use the VOLEitH approach to commit to $\hat{P}(X) := X \cdot P(1/X)$. If the witness is encoded as the constant term of P , then it is encoded as the leading term of $\hat{P}$. To open the evaluation $P(\Delta)$, the prover simply opens the evaluation $\hat{P}(\Delta_{\text{inv}})$ with $\Delta_{\text{inv}} := \Delta^{-1}$, which allows the verifier to retrieve $P(\Delta)$ as $P(\Delta) = \Delta \cdot \hat{P}(\Delta_{\text{inv}})$.

2.2 Overview of SD-in-the-Head-2

The SD-in-the-Head-2 scheme relies on the proof system described in Section 2.1, using the VOLE-in-the-Head approach, which enables proving knowledge of a witness satisfying a set of degree- d polynomial constraints. In what follows, **we explain how we express a syndrome decoding instance as such a system of degree- d constraints, in a way that aims for small witness size (and hence small signature size).** We then describe the optimized all-but-one vector commitment our scheme relies on. We finally address the transformation of the obtained 7-round interactive proof into a signature scheme using the standard Fiat-Shamir transformation.

SD modeling. We need to express a syndrome decoding instance as a system of degree- d constraints. To proceed, we will rely on the syndrome decoding modeling proposed by [OTX24; BBG⁺24]. The high-level idea is to consider a structured syndrome decoding problem, more precisely the regular syndrome decoding problem (RSD). However, as suggested in [BBG⁺24], *we use RSD parameters for which we have a provable security reduction to a secure unstructured syndrome decoding instance*, using the reduction from [FJR22]. Therefore, the security of SD-in-the-Head-2 still inherits from the conservative security of the oldest hard problem of code-based cryptography, namely the syndrome decoding problem for (unstructured) random linear codes.

Given a matrix $\mathbf{H} \in \mathbb{F}_2^{(n-k) \times n}$ and a syndrome vector $\mathbf{y} \in \mathbb{F}_2^{n-k}$, the RSD problem consists in finding a vector $\mathbf{x} \in \mathbb{F}_2^n$ such that

- $\mathbf{x}$ satisfies the linear relation $\mathbf{y} = \mathbf{H}\mathbf{x}$, and

- $\mathbf{x}$ is regular, meaning that it is the concatenation of w elementary vectors $e_1, \dots, e_w$ of size $\frac{n}{w}$ (i.e. vector with $\frac{n}{w} - 1$ coefficients 0 and one coefficient 1).

To express an RSD instance as a constraint system, [OTX24; BBG⁺24] proposes to rely on a compression version of $x := (e_1 \parallel \dots \parallel e_w)$: their idea consists in deriving each vector e_i as a tensor product of $\log_2(\frac{n}{w})$ elementary vectors of size 2:

$$e_i \leftarrow \begin{pmatrix} b_{i,1} \\ 1 - b_{i,1} \end{pmatrix} \otimes \begin{pmatrix} b_{i,2} \\ 1 - b_{i,2} \end{pmatrix} \otimes \dots \otimes \begin{pmatrix} b_{i,\log_2(n/w)} \\ 1 - b_{i,\log_2(n/w)} \end{pmatrix} \in \mathbb{F}^{n/w},$$

where $b_{i,1}, \dots, b_{i,\log_2(n/w)}$ is the binary decomposition of the non-zero positions of e_i . In that case, the j^{th} coordinate of e_i is derived as

$$(e_i)_j \leftarrow \prod_{k=1}^{\log_2(n/w)} (b_{i,k} \oplus \overline{\text{bit}_k(j)}),$$

where $\text{bit}_k(j)$ is the k^{th} bit of the integer j and $\overline{\cdot}$ is the negation function. By doing this, we do not need to check that the e_i 's are elementary since this property is guaranteed by design (i.e. a vector e_i satisfying the above equation is always an elementary vector, whatever are the values $b_{i,1}, \dots, b_{i,\log_2(n/w)}$). This construction leads to a witness of $w \cdot \log_2(\frac{n}{w})$ bits which is verified using a system of degree- d constraints, with $d = \log_2(\frac{n}{w})$. We thus obtain a small witness, but constraints of relatively high degree. We can relax [OTX24; BBG⁺24]'s idea with the following tweak. We build each elementary vector e_i as a tensor product of d elementary vectors of size $\mu_1, \dots, \mu_d$ (with $\mu_1 \cdot \mu_2 \cdot \dots \cdot \mu_d = n/w$). We obtain a witness of bit-size

$$\sum_{i=1}^d (\mu_i - 1),$$

and a system of degree- d constraints. By taking μ_i a bit larger than 2 (for example, 4), we only slightly increase the witness size, but significantly we reduce the constraints' degree. On the other hand, by using elementary vectors of size $\mu_i > 2$, we need to add further (degree-2) constraints to prove that the e_i 's are elementary vectors.

Specifically, the signer shall prove that they know bits $\{b_{i,j,k}\}_{1 \leq i \leq w, 1 \leq j \leq d, 1 \leq k \leq \mu_j - 1}$ such that

- we have $\mathbf{y} = \mathbf{H}\mathbf{x}$ with $x := (e_1 \parallel \dots \parallel e_w)$, where e_i is defined as

$$e_i \leftarrow \begin{pmatrix} b_{i,1,1} \\ b_{i,1,2} \\ \vdots \\ b_{i,1,\mu_1-1} \\ 1 - \sum_k b_{i,1,k} \end{pmatrix} \otimes \begin{pmatrix} b_{i,2,1} \\ b_{i,2,2} \\ \vdots \\ b_{i,2,\mu_2-1} \\ 1 - \sum_k b_{i,2,k} \end{pmatrix} \otimes \dots \otimes \begin{pmatrix} b_{i,d,1} \\ b_{i,d,2} \\ \vdots \\ b_{i,d,\mu_d-1} \\ 1 - \sum_k b_{i,d,k} \end{pmatrix} \in \mathbb{F}^{n/w}$$

for all $1 \leq i \leq w$;

- for all pairs (i, j) , $(b_{i,j,1}, \dots, b_{i,j,\mu_j})$ has at most one non-zero coordinate, implying that

$$\begin{pmatrix} b_{i,j,1} \\ b_{i,j,2} \\ \vdots \\ b_{i,j,\mu_j-1} \\ 1 - \sum_k b_{i,j,k} \end{pmatrix}$$

is an elementary vector.

AVC Optimizations. As explained in Section 2.1, the proof system starts by committing independently to τ vector polynomials $P^{(1)}, \dots, P^{(\tau)}$. It implies that we use τ all-but-one vector commitments, where each of them uses of GGM tree of N leaves. Therefore, we can optimize those all-but-one vector commitments by using a so-called batched all-but-one vector commitment (BAVC) scheme, which aims to be more efficient than τ independent AVC schemes. In SD-in-the-Head-2, we use the “one-tree” BAVC scheme described in [BBM⁺24]. Instead of considering τ independent GGM trees of N leaves in parallel, this scheme relies on a unique large GGM tree of $\tau \cdot N$ leaves where the i^{th} seed of the e^{th} parallel repetition is associated to the $(e \cdot N + i)^{\text{th}}$ leaf of the large GGM tree. As explained in [BBM⁺24], “opening all but τ leaves of the big tree is more efficient than opening all but one leaf in each of the τ smaller trees, because with high probability some of the active paths in the tree will merge relatively close to the leaves, which reduces the number of internal nodes that need to be revealed.” Moreover, the authors of [BBM⁺24] suggest improving the previous approach by rejecting certain bad challenges for which the paths did not sufficiently merged. When the BAVC opening is such that the number of revealed nodes in the sibling paths exceeds a fixed threshold T_{open} , the opening is considered as a failure (*i.e.* it returns $\perp$), forcing the prover/signer to recompute another opening challenge by re-hashing with an incremented counter. This process is done until the number of revealed nodes is less than T_{open} . For example, if we consider $N = 256$ and $\tau = 16$, the number of revealed nodes is smaller than (or equal to) $T_{\text{open}} := 110$ with probability ≈ 0.2 (which is to be compared to $\tau \cdot \log_2(N) = 128$ nodes for the sibling paths of separated trees). The selected value of T_{open} induces a rejection probability $p_{\text{rej}} = 1 - 1/\theta$, for some $\theta \in (0, \infty)$, hence the signer needs to perform an average of θ hash computations for the opening challenge (instead of 1). While this strategy decreases the challenge space by a factor θ , it does not change the average number of hashes that must be computed in a forgery attempt against the signature scheme (since the latter is multiplied by θ). As noticed by the authors of [BBM⁺24], this strategy can be thought of as losing $\log_2 \theta$ bit of security (because of a smaller challenge space) which are regained thanks to the implicit proof-of-work for finding a good challenge.

Fiat-Shamir transformation & grinding. To obtain the SD-in-the-Head-2 signature scheme from the SD-in-the-Head-2 zero-knowledge proof of knowledge, we rely on the Fiat-Shamir transformation [FS87] to remove the prover-verifier interactions. We further proceed with adding a random salt to enforce domain separation between signatures (and avoid seed collision issues). Each verifier challenge is computed as the output of an extendable-output function (XOF) which takes as input the data that the prover would send before receiving that challenge in an interactive protocol. The SD-in-the-Head-2 protocol is a 7-round proof system, so there are three challenges: the random matrix M of the consistency check, the random coefficients $\gamma_1, \dots, \gamma_m$ to batch the polynomial constraints, and the evaluation point Δ for the opened evaluations. In SD-in-the-Head-2 scheme, we use a grinding proof-of-work in the Fiat-Shamir hash computation of the last challenge, as proposed in [BBM⁺24]. Together with the opening challenge, the signer samples a w_{pow} -bit value v_{pow} and keeps the opening challenge only if this additional value is zero, with w_{pow} a parameter of the scheme. If this additional value is non-zero, then the signer increments a counter and recompute another opening challenge with another w_{pow} -bit value. This process is repeated until obtaining a grinding value equal to zero. Let us remark that we can use the same counter for this grinding process and the rejection process due to the fact that the [BBM⁺24]’s BAVC scheme might return $\perp$ when the number of revealed nodes is larger than the chosen threshold T_{open} . This grinding tweak increases the cost of hashing the



last challenge by a factor $2^{w_{\text{pow}}}$ and hence increases the soundness security of w_{pow} bits. As a result, we can take smaller parameters (N, τ) for the large tree, namely parameters achieving $\lambda - w_{\text{pow}}$ bits of security instead of λ . More precisely, the parameters N , τ and w_{pow} must be chosen such that $(d/N^\tau) \cdot 2^{-w_{\text{pow}}} \leq 2^{-\lambda}$ to achieve a λ -bit **security**.

3 Detailed algorithmic description

3.1 Main algorithms

3.1.1 Key generation

The key generation consists in sampling a regular syndrome decoding instance. The public key is the instance $(\mathbf{H}, \mathbf{y})$ while the secret key is composed of the instance and the associated solution $(\mathbf{H}, \mathbf{y}, \mathbf{x})$. Moreover, we consider that H is in standard form, *i.e.* $\mathbf{H} := (\mathbf{H}' \mid \mathbf{I}_{n-k})$.

We describe in Algorithm 1 the subroutine ExpandWitness which expands a λ -bit seed into a regular syndrome decoding solution $\mathbf{x}$ and builds the SD-in-the-Head-2 witness wit , where wit encodes the positions of the non-zero coefficients. To proceed, we first sample the position of the non-zero coefficients in each solution chunk of size m (Line 2) from which we derive the regular SD solution $\mathbf{x}$. Then we build the witness by encoding the positions. As explained in Section 2.2, we write each chunk as the tensor product of d elementary vectors of size $\mu_1, \dots, \mu_d$. We omit the last coordinate of all the subvectors (Line 11), since it can be deduced from the others.

Algorithm 1 ExpandWitness

Input: a seed seed

```

    ▷ Expand the RSD solution
1:  $\text{prg} \leftarrow \text{PRG.Init}(\text{seed})$ 
2:  $\text{pos} \leftarrow \text{SampleIntegers}(\text{prg}, \{0, \dots, m-1\}, w)$            ▷  $\text{pos} \in \{0, \dots, m-1\}^w$ 
3: for  $i$  from 0 to  $w-1$  do
4:    $\text{chunk}[i] := \text{ElementaryVector}(m, \text{pos}[i])$ 
5:  $\mathbf{x} \leftarrow (\text{chunk}[0] \parallel \dots \parallel \text{chunk}[w-1])$            ▷  $\mathbf{x} \in \mathbb{F}_q^n$ 

    ▷ Build the witness
6:  $\text{wit} \leftarrow \emptyset$ 
7: for  $i$  from 0 to  $w-1$  do
8:   for  $j$  from 0 to  $d-1$  do
9:      $(\text{pos}[i], p) \leftarrow (\text{pos}[i] // \mu_j, \text{pos}[i] \% \mu_j)$ 
10:     $\mathbf{e} := (e_0, \dots, e_{\mu_j-1}) \leftarrow \text{ElementaryVector}(\mu_j, p)$ 
11:     $\text{wit} \leftarrow \text{wit} \parallel (e_0, \dots, e_{\mu_j-2})$ 
12: return  $(\mathbf{x}, \text{wit})$ 

```

The key generation is described in Algorithm 2. It first samples two seeds seed_{sk} and seed_{pk} . The first seed seed_{sk} is used to expand the syndrome decoding solution $\mathbf{x}$ and the SD-in-the-Head witness wit through the subroutine ExpandWitness. The second seed seed_{pk} is used to generate the matrix $\mathbf{H}'$ which defines the parity check matrix $\mathbf{H} := (\mathbf{H}' \mid \mathbf{I}_{n-k})$. The algorithm then builds the public key by packing the seed seed_{pk} which encodes $\mathbf{H}$ and the vector $\mathbf{y} := \mathbf{H}\mathbf{x}$. It also builds the secret key by packing the seed seed_{sk} , the vector $\mathbf{y}$ and the SD-in-the-Head-2 witness wit .

Algorithm 2 SD-in-the-Head-2 – Key Generation

```

1:  $\text{seed}_{\text{sk}} \leftarrow \{0, 1\}^\lambda$ 
2:  $\text{seed}_{\text{pk}} \leftarrow \{0, 1\}^\lambda$ 
3:  $(\mathbf{x}, \text{wit}) \leftarrow \text{ExpandWitness}(\text{seed}_{\text{sk}})$   $\triangleright x \in \mathbb{F}_q^n, |\text{wit}|_2 = w \cdot ((\mu_1 - 1) + \dots + (\mu_d - 1))$ 
4:  $\mathbf{H}' \leftarrow \text{ExpandH}(\text{seed}_{\text{pk}})$   $\triangleright \mathbf{H}' \in \mathbb{F}_q^{(n-k) \times k}$ 
5:  $\mathbf{y} = [\mathbf{H}' \mid \mathbf{I}_{n-k}] \cdot \mathbf{x}_A$   $\triangleright \mathbf{y} \in \mathbb{F}_q^{n-k}$ 
6:  $\text{pk} = \text{Serialize}(\text{seed}_{\text{pk}}, \mathbf{y})$ 
7:  $\text{sk} = \text{Serialize}(\text{seed}_{\text{pk}}, \mathbf{y}, \text{wit}, \text{seed}_{\text{sk}})$ 
8: return (pk, sk)

```

3.1.2 Signing

The signing algorithm is described in Algorithm 3. After expanding the secret key as $(\mathbf{H}', \mathbf{y}, \text{wit})$, it samples a λ -bit salt which provides domain separation between signatures and a λ -bit root seed from which all the pseudo-randomness of the scheme will be derived. Then, using the routine **BLC.Commit**, it commits to random degree-1 vector polynomials $\mathbf{P}_{\text{wit}}(X) \in (\mathbb{F}_{2^\lambda}[X])^{|\text{wit}|}$ and $\mathbf{P}_{\text{rnd}}(X) \in (\mathbb{F}_{2^\lambda}[X])^{(d-1)\lambda}$ such that $\mathbf{P}_{\text{wit}}(0) = \text{wit}$ and $\mathbf{P}_{\text{rnd}}(0)$ is uniformly sampled from $\mathbb{F}_2^{(d-1)\lambda}$. While $\mathbf{P}_{\text{wit}}$ encodes the witness in the PIOP protocol, $\mathbf{P}_{\text{rnd}}$ aims to prevent witness leakage in the protocol. The hash digest h_{lines} is the BLC commitment to these polynomials. After the commitment phase, it runs the PIOP protocol using the routine **PIOP.Prover.Run**, namely it computes the degree- d polynomial P_α such that

$$P_\alpha(X) = P_0(X) \cdot X + \sum_{j=1}^m \gamma_j \cdot f_j(P_1(X), \dots, P_{|\text{wit}|}(X)).$$

where

- $P_1, \dots, P_{|\text{wit}|}$ are the witness polynomial, *i.e.* $\mathbf{P}_{\text{wit}} := (P_1, \dots, P_{|\text{wit}|})$;
- P_0 is the degree- $(d-1)$ masking polynomial built as

$$P_0(X) := \sum_{i=1}^{d-1} \left(\sum_{j=1}^{\lambda} \delta^{j-1} \cdot P_{\text{rnd}, i, j}(X) \right) \cdot X^{i-1}$$

with $\mathbf{P}_{\text{rnd}} := (P_{\text{rnd}, 1, 1}, \dots, P_{\text{rnd}, 1, \lambda}, \dots, P_{\text{rnd}, d-1, 1}, \dots, P_{\text{rnd}, d-1, \lambda})$ and $(1, \delta, \delta^2, \dots)$ is a $\mathbb{F}_2$ -basis of $\mathbb{F}_{2^\lambda}$;

- $\{f_j\}_j$ are the degree- d polynomial constraints that the SD-in-the-Head-2 witness wit should satisfy (*c.f.* Section 2.2).

By design, P_α satisfies $P_\alpha(0) = 0$, so we can write P_α as $\sum_{i=1}^d \alpha_i \cdot X^i$ (*i.e.* without constant term). Then, the signing algorithm hashes P_α (by hashing its coefficients), samples a random evaluation point $\Delta \in \mathbb{F}_{2^\lambda}$ and opens the polynomial evaluations $\mathbf{P}_{\text{wit}}(\Delta)$ and $\mathbf{P}_{\text{rnd}}(\Delta)$ using the routine **BLC.OpenRandomEvaluation**. By “opening”, we mean that the signer provides an opening data **pdecom** that enables the verifier to recover $\mathbf{p}_{\text{wit}} := \mathbf{P}_{\text{wit}}(\Delta)$ and $\mathbf{p}_{\text{rnd}} := \mathbf{P}_{\text{rnd}}(\Delta)$. The verifier can then check that those evaluations are consistent with the BLC commitment h_{lines} .

Algorithm 3 SD-in-the-Head-2 – **Signing Algorithm****Input:** a secret key $\mathbf{sk}$ and a message $\mathbf{msg} \in \{0, 1\}^*$

▷ Phase 0: Initialization.

1: $(\text{seed}_{\mathbf{pk}}, \mathbf{y}, \text{wit}, \text{seed}_{\mathbf{sk}}) \leftarrow \text{Deserialize}(\mathbf{sk}, \{0, 1\}^\lambda, \mathbb{F}_q^{n-k}, \mathbb{F}_2^{|\text{wit}|})$ 2: $\mathbf{H}' \leftarrow \text{ExpandH}(\text{seed}_{\mathbf{pk}})$ ▷ $\mathbf{H}' \in \mathbb{F}_q^{(n-k) \times k}$ 3: $\text{salt} \leftarrow \{0, 1\}^\lambda$ 4: $\text{rseed} \leftarrow \{0, 1\}^\lambda$ 

▷ Phase 1: Build & Commit Witness/Masking Polynomials.

5: $(\mathbf{P}_{\text{wit}}, \mathbf{P}_{\text{rnd}}, h_{\text{lines}}, \text{key}, \text{aux}) \leftarrow \text{BLC.Commit}(\text{salt}, \text{rseed}, \text{wit})$ ▷ $\mathbf{P}_{\text{wit}}(0) = \text{wit}$

▷ Phase 2: PIOP Protocol (prover side).

6: $P_\alpha \leftarrow \text{PIOP.Prover.Run}(\mathbf{P}_{\text{wit}}, \mathbf{P}_{\text{rnd}}, h_{\text{lines}}, (\mathbf{H}', \mathbf{y}))$ ▷ $P_\alpha(X) \in \mathbb{F}_{2^\lambda}^{(\leq d)}[X]$ 7: $h_{\text{piop}} \leftarrow \text{Hash}_{\text{piop}}(\mathbf{pk}, h_{\text{lines}}, \alpha_1, \dots, \alpha_d, \text{msg})$ ▷ $P_\alpha(X) = \sum_{0 \leq i \leq d} \alpha_i \cdot X^i$

▷ Phase 3: Open random evaluations.

8: $\text{pdecom} \leftarrow \text{BLC.OpenRandomEvaluation}(\text{key}, h_{\text{piop}})$ 9: $\sigma = \text{Serialize}(\text{salt} \parallel h_{\text{piop}} \parallel \text{aux} \parallel \text{pdecom} \parallel (\alpha_1, \alpha_2, \dots, \alpha_d))$ 10: **return** σ **3.1.3 Verification**

The verification algorithm is described in **Algorithm 4**. After expanding the public key as $(\mathbf{H}', \mathbf{y})$ and parsing the signature, it recovers the evaluations $\mathbf{p}_{\text{wit}} := \mathbf{P}_{\text{wit}}(\Delta)$ and $\mathbf{p}_{\text{rnd}} := \mathbf{P}_{\text{rnd}}(\Delta)$ from pdecom as well as h_{lines} using the routine **BLC.RecomputeEvaluation**. If the recovered h_{lines} does not match the value of the signing algorithm, the later verification $h_{\text{piop}} = h'_{\text{piop}}$ will fail with overwhelming probability, since h_{lines} is input of the hash computation of h'_{piop} . At the same time, **BLC.RecomputeEvaluation** outputs the random evaluation point Δ . The verification algorithm then deduces the evaluation $p_\alpha = P_\alpha(\Delta)$ using the routine **PIOP.ComputeOutput** and check that it is consistent with the polynomial P_α included in the signature. Finally, it checks that the polynomial P_α is the same as in the signing algorithm by comparing the hash digests.

Algorithm 4 SD-in-the-Head-2 – Verification Algorithm

Input: a public key $\mathbf{pk}$, a signature σ and a message $\mathbf{msg} \in \{0, 1\}^*$

▷ Phase 0: Initialization.

1: $(\text{salt} \parallel h_{\text{piop}} \parallel \text{aux} \parallel \text{pdecom} \parallel (\alpha_1, \alpha_2, \dots, \alpha_d)) \leftarrow \text{Deserialize}(\sigma)$

2: $(\text{seed}_{\mathbf{pk}}, \mathbf{y}) \leftarrow \text{Deserialize}(\mathbf{pk})$

3: $\mathbf{H}' \leftarrow \text{ExpandH}(\text{seed}_{\mathbf{pk}})$

▷ $\mathbf{H}' \in \mathbb{F}_q^{(n-k) \times k}$

▷ Phase 1: Recomputing Evaluation.

4: $(\Delta, \mathbf{p}_{\text{wit}}, \mathbf{p}_{\text{rnd}}, h_{\text{lines}}) \leftarrow \text{BLC.RecomputeEvaluation}(\text{aux}, \text{pdecom}, \text{salt}, h_{\text{piop}})$

5:

▷ $\mathbf{p}_{\text{wit}} := \mathbf{P}_{\text{wit}}(\Delta), \mathbf{p}_{\text{rnd}} := \mathbf{P}_{\text{rnd}}(\Delta)$

6: **if** $\Delta = \perp$ **then**

7: **return** REJECT

▷ Phase 2: PIOP Protocol (verifier side).

8: $p_\alpha \leftarrow \text{PIOP.Verifier.Run}(\Delta, \mathbf{p}_{\text{wit}}, \mathbf{p}_{\text{rnd}}, h_{\text{lines}}, (\mathbf{H}', \mathbf{y}))$

9: $h'_{\text{piop}} = \text{Hash}_{\text{piop}}(\mathbf{pk}, h_{\text{lines}}, \alpha_1, \dots, \alpha_d, \mathbf{msg})$

▷ $p_\alpha := P_\alpha(\Delta)$
 ▷ $P_\alpha(X) = \sum_{0 \leq i \leq d} \alpha_i \cdot X^i$

▷ Phase 3: Verification.

10: **if** $h_{\text{piop}} \neq h'_{\text{piop}}$ or $p_\alpha \neq \sum_{i=1}^d \alpha_i \cdot \Delta^i$ **then**

11: **return** REJECT

12: **return** ACCEPT

3.2 Subroutines

3.2.1 Symmetric cryptography primitives

The SD-in-the-Head-2 signature scheme relies on three types of symmetric cryptography primitives: a hash function (Hash), and an extendable output function (XOF), and a block cipher (Enc). The instantiations of these primitives are summarized in Table 1.

Table 1: Symmetric cryptography primitives for NIST Security Categories I, III, and V. For Category III, the block cipher is defined as a truncated version of Rijndael-256-256.

	Category I	Category III	Category V
Enc	AES-128	Rijndael-256-256*	Rijndael-256-256
Hash	SHAKE-128	SHAKE-256	SHAKE-256
XOF	SHAKE-128	SHAKE-256	SHAKE-256

3.2.2 Pseudo-randomness generation

Several subroutines used in the SD-in-the-Head-2 signature schemes involve pseudorandomness generation from a seed. Several seeds are expanded from a master seed in the key generation and in the signature algorithm (to generate the lines). One also needs to sample sequences of field elements from a seed in the key generation, the signature and verification algorithms. Finally pseudorandomness generation is also involved to derive the challenges (consistency challenge, batching challenge, and evaluation challenge) from the Fiat-Shamir hashes h_{aux} , h_{lines} and h_{piop} .

Pseudo-random generator. Most the pseudorandomness (everything except the Fiat-Shamir challenges) in SD-in-the-Head-2 is generated through a pseudo-random generator (PRG). Such a function takes a λ -bit seed $\text{seed} \in \{0, 1\}^\lambda$ and produces an arbitrary-long output bit-string $y \in \{0, 1\}^*$ whose length is tailored to the requirements of the application. Formally, a PRG is equipped with two routines: $\text{PRG.Init}(x)$ initializes the PRG state with the input $x \in \{0, 1\}^*$. Once initialized, the PRG can be queried with the routine $\text{PRG.GetByte}()$ to generate the next byte of the output y associated to x . In our context, we use a block cipher Enc in counter mode as a secure pseudorandom generator (PRG). The concrete instance of the block cipher we use in the SD-in-the-Head-2 scheme is given in Section 3.2.1. The initialization function $\text{PRG.Init}(x, \text{salt})$ might take an additional input salt, which will correspond to the nonce for the counter mode. Regarding Category III, since we use Rijndael-256-256 as mentioned in Section 3.2.1, we pad the input seed and salt with 64 least significant zero bits, and the output of the PRG corresponds to the output blocks of the cipher (without any truncation).

Extendable output function. The pseudorandomness in SD-in-the-Head-2 for Fiat-Shamir challenges is generated through an extendable output hash function (XOF). Such a function takes an arbitrary-long input bit-string $x \in \{0, 1\}^*$ and produces an arbitrary-long output bit-string $y \in \{0, 1\}^*$ whose length is tailored to the requirements of the application. Formally, a XOF is equipped with two routines: $\text{XOF.Init}(x)$ initializes the XOF state with the input $x \in \{0, 1\}^*$. Once initialized, the XOF can be queried with the routine $\text{XOF.GetByte}()$ to generate the next byte of the output y associated to x . The concrete instance of the XOF we use

in the SD-in-the-Head-2 scheme is given in Section 4.3. In our context, we use the XOF as a secure pseudorandom generator (PRG) which tolerates input seeds of variable lengths.

Sampling (a vector of) field elements. The subroutine `SampleFieldElements(src,  $\mathbb{F}$ ,  $n$ )` samples n pseudorandom elements from $\mathbb{F}$ using the PRG/XOF `src`. It assumes that the XOF/PRG has been previously initialized. The implementation of the `SampleFieldElements` routine use the following process. It first generates from `src` a stream of bytes $B_1, \dots, B_{n'}$ for some $n' \geq (n \cdot \log_2 |\mathbb{F}|)/8$. Those bytes are converted into n field elements as follows:

- For $\mathbb{F}_q = \mathbb{F}_2$: The byte B_i is interpreted as 8 field elements $b_{i,0}, \dots, b_{i,7}$, such that $B_i = \sum_j 2^j \cdot b_{i,j}$. Interpreted all the bytes leads to a vector $(b_{1,0}, \dots, b_{1,7}, \dots, b_{n',0}, \dots, b_{n',7})$ and the procedure returned the n first coordinates as the sampled field elements.
- For $\mathbb{F}_q = \mathbb{F}_{2^\lambda}$: The i^{th} pack of $\frac{\lambda}{8}$ consecutive bytes is returned as the i^{th} sampled field element.

Sampling integers. The subroutine `SampleIntegers(src,  $\{0, \dots, m-1\}$ ,  $n$ )` samples n pseudorandom integers from $\{0, \dots, m-1\}$ using the XOF/PRG `src`, where $m \leq 2^{32}$. It assumes that the XOF/PRG has been previously initialized. The implementation of the `SampleIntegers` routine uses the principle of rejection sampling. While denoting $t_{\max}$ as the largest multiple of m smaller then (or equal to) 2^{32} , the procedure goes as follows:

```

1:  $i = 1$ 
2: while  $i \leq n$  do
3:   for  $0 \leq j < 4$  do
4:      $B_j \leftarrow \text{src.GetByte}()$ 
5:      $v \leftarrow B_0 + 256 \cdot B_1 + 256^2 \cdot B_2 + 256^3 \cdot B_3$ 
6:     if  $v \in \{0, 1, \dots, t_{\max} - 1\}$  then
7:        $f_i = B \% m$ ;  $i++$ 
8: return  $(f_1, \dots, f_n)$ 

```

The number of generated bytes which are necessary to complete the process is non-deterministic.

Expansion of the parity-check matrix. The subroutine `ExpandH` takes as input λ -bit seed `seedH` and returns an $(n-k) \times k$ matrix of elements of $\mathbb{F}_q$. This generated matrix is the random part H' of the parity-check matrix in standard form $H = (H' | I_{n-k})$. A call to `ExpandH(seedH)` generates H' row-wise as follows:

```

1:  $\text{prg} \leftarrow \text{PRG.Init}(\text{seed}_H)$ 
2: for  $i$  from 0 to  $k-1$  do
3:    $\text{cols}_{H'}[i] \leftarrow \text{SampleFieldElements}(\text{prg}, \mathbb{F}_q, n-k)$ 
4:  $H' \leftarrow [\text{cols}_{H'}[0] \mid \dots \mid \text{cols}_{H'}[k-1]]$   $\triangleright H' \in \mathbb{F}_2^{(n-k) \times k}$ 
5: return  $H'$ 

```

Seed expansion (GGM Tree). The subroutine `ExpandSeed` expands a salt, a parent seed and an index into two seeds. It is used to expand GGM trees. Specifically, a call to `ExpandSeed(salt, seed, idx)` runs the following procedure for Categories I and V:

```

1: left  $\leftarrow \text{Enc}_\lambda(\text{key} = \text{seed}, \text{ptx} = \text{salt} \oplus \text{MapToBits}(2 \cdot \text{idx}))$ 
2: right  $\leftarrow \text{Enc}_\lambda(\text{key} = \text{seed}, \text{ptx} = \text{salt} \oplus \text{MapToBits}(2 \cdot \text{idx} + 1))$ 
3: return (left, right)

```

where `MapToBits` computes the bitstring that corresponds to the binary decomposition of the input integer in a little-endian order. For Category III, a call to `ExpandSeed(salt, seed, idx)` runs as follows:

```

1: left  $\leftarrow \text{Rijndael-256-256}(\text{key} = (0^{64} \parallel \text{seed}), \text{ptx} = (0^{64} \parallel \text{salt}) \oplus \text{MapToBits}(2 \cdot \text{idx}))$ 
2: right  $\leftarrow \text{Rijndael-256-256}(\text{key} = (0^{64} \parallel \text{seed}), \text{ptx} = (0^{64} \parallel \text{salt}) \oplus \text{MapToBits}(2 \cdot \text{idx} + 1))$ 
3: return ( $\text{GetMSB}_{192}(\text{left})$ ,  $\text{GetLSB}_{192}(\text{right})$ )

```

where GetMSB_{192} and GetLSB_{192} return respectively the 192 most significant bits and the 192 less significant bits.

Expansion of consistency check challenge. The subroutine `ExpandConsistencyChallenge` expands the first Fiat-Shamir hash h_{aux} into the matrix $\mathbf{M}$ used for the consistency check. It consists of the following steps:

```

1: xof  $\leftarrow \text{XOF.Init}(h_{\text{aux}})$ 
2: for  $i$  from 0 to  $\ell - 1$  do
3:    $\text{cols}_{\mathbf{M}}[i] \leftarrow \text{SampleFieldElements}(\text{xof}, \mathbb{F}_2, \lambda + B)$ 
4:  $\mathbf{M} \leftarrow [\text{cols}_{\mathbf{M}}[0] \mid \dots \mid \text{cols}_{\mathbf{M}}[\ell - 1]]$   $\triangleright \mathbf{M} \in \mathbb{F}_2^{(\lambda+B) \times \ell}$ 
5: return  $\mathbf{M}$ 

```

Expansion of batching challenge. The subroutine `ExpandBatchingChallenge` expands the second Fiat-Shamir hash h_{lines} into the batching challenges (γ', γ) . It consists of the following steps:

```

1: xof  $\leftarrow \text{XOF.Init}(h_{\text{lines}})$ 
2:  $\gamma' \leftarrow \text{SampleFieldElements}(\text{xof}, \mathbb{F}_{2^\lambda}, w \cdot d')$ , where  $d' = \#\{j : \mu_j > 2\}$ 
3:  $\gamma \leftarrow \text{SampleFieldElements}(\text{xof}, \mathbb{F}_{2^\lambda}, \lceil \frac{n-k}{\lambda} \rceil)$ 
4: return  $(\gamma', \gamma)$ 

```

Expansion of evaluation-opening challenge. The subroutine `ExpandEvaluationChallenge` expands the third Fiat-Shamir hash h_{piop} into the evaluation-opening challenges $i^*[1], \dots, i^*[\tau]$, where $i^*[e]$ is the hidden seed which should remain hidden for execution e . It also expands a w_{pow} -bit grinding digest v_{pow} , which will lead to a challenge rejection when it is non-zero. This subroutine takes as input the hash h_{piop} and a 32-bit counter ctr . It consists of the following steps:

```

1: xof  $\leftarrow \text{XOF.Init}(h_{\text{piop}}, \text{ctr})$ 
2:  $\{i^*[e]\}_{e < \tau} \leftarrow \text{SampleIntegers}(\text{xof}, \{0, \dots, 2^\kappa - 1\}, \tau)$ 
3:  $v_{\text{pow}} \leftarrow \text{SampleFieldElements}(\text{xof}, \mathbb{F}_2, w_{\text{pow}})$ 
4: return  $(\{i^*[e]\}_{e < \tau}, v_{\text{pow}})$ 

```

3.2.3 Hashing and commitments

Several subroutines used in the SD-in-the-Head signature scheme involve cryptographic hashing. This is the case of the subroutines computing the Fiat-Shamir hashes and the commitments. We also use a cryptographic hash function for the seed trees (hypercube variant) and the Merkle trees (threshold variant).

Cryptographic hash function. The different hash and commitment subroutines are all derived from a common cryptographic hash function

$$\text{Hash} : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}.$$

The concrete instance of the hash function we use in the SD-in-the-Head scheme is given in Section 4.3.

We use domain separation for the different usages of the hash function. This is simply done by prepending a fixed byte value to the data to be hashed, as specified below for the different cases.

Seed Commitments. The subroutine **CommitSeed** takes as input a λ -bit salt, an index idx and a λ -bit seed, and it outputs a commitment digest for this seed. For performance reason, while we used hashing to commit to seeds in the first version of SD-in-the-Head, we use a block cipher in the second version. For Categories I and V, we build the commitment digest com for the seed seed as follows:

- 1: $\text{tweak} \leftarrow 2 \cdot (\tau \cdot N + \text{idx})$
- 2: $\text{left} \leftarrow \text{Enc}_\lambda(\text{key} = \text{seed}, \text{ptx} = \text{salt} \oplus \text{MapToBits}(\text{tweak}))$
- 3: $\text{right} \leftarrow \text{Enc}_\lambda(\text{key} = \text{seed}, \text{ptx} = \text{salt} \oplus \text{MapToBits}(\text{tweak} + 1))$
- 4: $\text{com} \leftarrow (\text{left} \parallel \text{right})$ $\triangleright \text{com} \in \{0, 1\}^{2\lambda}$
- 5: **return** com

where **MapToBits** computes the bitstring that corresponds to the binary decomposition of the input integer in a little-endian order. For Category III, we build the commitment digest as follows:

- 1: $\text{tweak} \leftarrow 2 \cdot (\tau \cdot N + \text{idx})$
- 2: $\text{left} \leftarrow \text{Rijndael-256-256}(\text{key} = 0^{64} \parallel \text{seed}, \text{ptx} = (0^{64} \parallel \text{salt}) \oplus \text{MapToBits}(\text{tweak}))$
- 3: $\text{right} \leftarrow \text{Rijndael-256-256}(\text{key} = 0^{64} \parallel \text{seed}, \text{ptx} = (0^{64} \parallel \text{salt}) \oplus \text{MapToBits}(\text{tweak} + 1))$
- 4: $\text{com} \leftarrow (\text{GetMSB}_{192}(\text{left}) \parallel \text{GetLSB}_{192}(\text{right}))$ $\triangleright \text{com} \in \{0, 1\}^{2\lambda}$
- 5: **return** com

where GetMSB_{192} and GetLSB_{192} return respectively the 192 most significant bits and the 192 less significant bits.

Fiat-Shamir Hashes. The hash functions used to derive the Fiat-Shamir Hashes are defined as:

$$\begin{aligned} \text{Hash}_{\text{bavc}}(\text{data}) &:= \text{Hash}(1 \parallel \text{data}) \\ \text{Hash}_{\text{aux}}(\text{data}) &:= \text{Hash}(2 \parallel \text{data}) \\ \text{Hash}_{\text{lines}}(\text{data}) &:= \text{Hash}(3 \parallel \text{data}) \\ \text{Hash}_{\text{piop}}(\text{data}) &:= \text{Hash}(4 \parallel \text{data}) \end{aligned}$$

where the prefixes 1, 2, 3 and 4 are encoded on one byte.

3.2.4 All-but-one vector commitments

As the first version of SD-in-the-Head, SD-in-the-Head-2 relies on an *all-but-one vector commitment* (AVC). This primitive commits N random λ -bit seeds and later opens/reveals all of them except one. In practice, since we need τ such sets of N seeds, we will rely on the *batched* variant leveraging the one-tree optimisation from [BBM⁺24].

Commitment routine. The commitment routine of the batched all-but-one vector commitment (BAVC) scheme is described in Algorithm 5. After expanding a large GGM tree of $\tau \cdot N$ leaves using the routine **ExpandSeed**, it commits each seed using the routine **CommitSeed** where the $(j+1)^{\text{th}}$ seed of the $(i+1)^{\text{th}}$ repetition is the $(i \cdot \tau + e + 1)^{\text{th}}$ leaf of the big tree. Then it hashes all the seed commitments and the resulting digest h_{com} forms the global commitment of all the seeds. It outputs the tree and the seed commitments as the opening key.

Algorithm 5 BAVC.Commit

Input: a salt $\text{salt} \in \{0, 1\}^\lambda$ and a root seed $\text{rseed} \in \{0, 1\}^\lambda$

- ▷ Expand the GGM tree
- 1: $\text{tree}[1] \leftarrow \text{rseed}$
- 2: **for** i from 1 to $(\tau \cdot N - 1)$ **do**
- 3: $(\text{tree}[2i], \text{tree}[2i + 1]) \leftarrow \text{ExpandSeed}(\text{salt}, \text{tree}[i], i)$
- ▷ Commit the seeds
- 4: **for** e from 0 to $(\tau - 1)$ **do**
- 5: **for** i from 0 to $(N - 1)$ **do**
- 6: $\text{seeds}[e][i] \leftarrow \text{tree}[\tau \cdot N + (i \cdot \tau + e)]$
- 7: $\text{commit}[e][i] \leftarrow \text{CommitSeed}(\text{salt}, \text{seeds}[e][i], i \cdot \tau + e)$
- ▷ Set commitment and key
- 8: $h_{\text{com}} \leftarrow \text{Hash}_{\text{bavc}}(\{\text{commit}[e][i]\}_{e,i})$
- 9: $\text{key} \leftarrow \text{tree} \parallel \text{commit}$
- 10: **return** $(\text{seeds}, h_{\text{com}}, \text{key})$

Opening routine. The opening routine of the BAVC scheme is described in Algorithm 6. It takes as input the opening key (the tree nodes and the seed commitments) and the list of the seeds to remain hidden. It outputs the opening BAVC proof π_{BAVC} , which will enable the verifier to recompute all the revealed leaves while checking their consistency with the BAVC commitment h_{com} . To proceed, the routine first searches the smallest set of tree nodes that enables to recompute all the leaves excluding those in the input list. This search algorithm involves a queue structure which comes with four dedicated subroutines:

- **Queue.Init()** returns a empty queue,
- **Queue.Enqueue(v)** pushes a value v at the end of the queue,
- **Queue.Dequeue()** pops the value which is at the top of the queue, and
- **Queue.Head()** returns the value which is at the top of the queue *without removing it*.

This routine follows the paths between the hidden leaves and the tree root, and at each intermediary node, decides whether the sibling node should be in the output set. If the number of revealed nodes is larger than T_{open} , the routine aborts. At the end of the search algorithm, it pads the computed sibling paths with zeroes such that the result is always of $T_{\text{open}} \cdot \lambda$ bits. Finally, it outputs the opening BAVC proof π_{BAVC} made of the sibling paths and the commitments of all the hidden leaves.

Algorithm 6 *BAVC.Open*

Input: a BAVC key key and a set $\{i^*[e]\}_{e < \tau}$ of τ indexes in $\{0, \dots, N - 1\}$

```

1:  $(\text{tree}, \text{commit}) \leftarrow \text{key}$ 
2:  $\text{queue} \leftarrow \text{Queue.Init}()$ 
3: for  $\text{idx} \in \{i^*[e] \cdot \tau + e : e \in \{0, \dots, \tau - 1\}\}$  in the decreasing order do
4:    $\text{queue.Enqueue}(\tau \cdot N + \text{idx})$ 
5:  $\text{path} \leftarrow \emptyset$ 
6: while  $\text{queue.Head}() \neq 1$  do ▷ While the queue head is not the root
7:    $\text{node\_idx} \leftarrow \text{queue.Dequeue}()$ 
8:   if  $|\text{queue}| \geq 2$  then
9:      $\text{sibling\_idx} = \text{node\_idx} \oplus 1$ 
10:    if  $\text{queue.Head}() = \text{sibling\_idx}$  then ▷ Check if the queue head is the sibling node
11:       $\text{queue.Dequeue}()$ 
12:    else if  $|\text{path}| < T_{\text{open}} \cdot \lambda$  then ▷ Check if the sibling path is not too long
13:       $\text{path} \leftarrow \text{path} \parallel \text{tree}[\text{sibling\_idx}]$  ▷ Append the sibling node to the path
14:    else
15:      return  $\perp$  ▷ Return failure since the sibling path is too long
16:     $\text{queue.Enqueue}(\lfloor \text{node\_idx} / 2 \rfloor)$ 
17:  $\text{path} \leftarrow \text{PadWithZero}(\text{path}, T_{\text{open}} \cdot \lambda)$  ▷  $\text{path} \in \{0, 1\}^{T_{\text{open}} \cdot \lambda}$ 
18:  $\text{commit}_{i^*} \leftarrow (\text{commit}[0][i^*[0]], \dots, \text{commit}[\tau - 1][i^*[\tau - 1]])$  ▷  $\text{commit}_{i^*} \in \{0, 1\}^{\tau \cdot (2\lambda)}$ 
19:  $\pi_{\text{BAVC}} \leftarrow (\text{path}, \text{commit}_{i^*})$ 
20: return  $\pi_{\text{BAVC}}$ 

```

Reconstruction routine. The reconstruction routine of the BAVC scheme is described in Algorithm 7. It takes as input the list of hidden seeds, the BAVC opening proof π_{BAVC} (containing the sibling paths and the commitments of the hidden seeds) and the salt. Using the same search algorithm as in the opening routine, it prefills the tree with the nodes in the sibling paths. It checks whether the used padding is valid (*i.e.* the padded bits are only zeroes). The routine then expands the GGM tree using the prefilled nodes and recomputes the commitments of all the revealed seeds. Finally, it recomputes the BAVC commitment h_{com} and returns it together with the revealed seeds.

Algorithm 7 BAVC.Reconstruct

Input: a set $\{i^*[e]\}_{e < \tau}$ of τ indexes in $\{0, \dots, N - 1\}$, a BAVC opening π_{BAVC} and a salt $\text{salt} \in \{0, 1\}^\lambda$

▷ Prefill the partial GGM tree

```

1: (path, commiti*) ←  $\pi_{\text{BAVC}}$ 
2: queue ← Queue.Init()
3: for idx ∈  $\{i^*[e] \cdot \tau + e : e \in \{0, \dots, \tau - 1\}\}$  in the decreasing order do
4:   queue.Enqueue( $\tau \cdot N + \text{idx}$ )
5: tree[1], ..., tree[ $2 \cdot \tau \cdot N - 1$ ] ←  $\emptyset, \dots, \emptyset$ 
6: while queue.Head() ≠ 1 do                                ▷ While the queue head is not the root
7:   node_idx ← queue.Dequeue()
8:   if |queue| ≥ 2 then
9:     sibling_idx = node_idx ⊕ 1
10:    if queue.Head() = sibling_idx then                        ▷ Check if the queue head is the sibling node
11:      queue.Dequeue()
12:    else if |path| <  $T_{\text{open}} \cdot \lambda$  then                      ▷ Check if the sibling path is not too long
13:      (tree[sibling_idx], path) ← path                          ▷ Extract the  $\lambda$  first bits of path
14:    else
15:      return ⊥                                                  ▷ Return failure since the sibling path is too long
16:    queue.Enqueue( $\lfloor \text{node\_idx} / 2 \rfloor$ )
17: if |path| > 0 and path ≠ 0 then                                ▷ Check that the padding is correct
18:   return ⊥

```

▷ Expand the partial GGM tree

```

19: for i from 1 to ( $\tau \cdot N - 1$ ) do
20:   if tree[i] ≠  $\emptyset$  then
21:     (nodes[2i], nodes[2i + 1]) ← ExpandSeed(salt, nodes[i], i)

```

▷ Recompute commitment

```

22: for e from 0 to ( $\tau - 1$ ) do
23:   for i from 0 to ( $N - 1$ ) do
24:     if i ≠ i*[e] then
25:       seeds[e][i] ← nodes[ $\tau \cdot N + (i \cdot \tau + e)$ ]
26:       com[e][i] ← CommitSeed(salt, seeds[e][i], i ·  $\tau + e$ )
27:     else
28:       seeds[e][i] ←  $\emptyset$ 
29:       (com[e][i] || commiti*) ← commiti*
30: hcom ← HashBAVC({com[e][i]}e,i)
31: return (hcom, seeds)

```

3.2.5 Batch line commitment

As explained in Section 2.1, the prover in the PIOP protocol of SD-in-the-Head-2 needs to commit degree-1 vector polynomials $\mathbf{P}_{\text{wit}} := (P_1, \dots, P_{|\text{wit}|}) \in (\mathbb{F}_{2^\lambda}[X])^{|\text{wit}|}$ such that $\mathbf{P}_{\text{wit}}(0) = \text{wit} \in \mathbb{F}_2^{|\text{wit}|}$, and $\mathbf{P}_{\text{rnd}} := (P_{\text{rnd},1,1}, \dots, P_{\text{rnd},1,\lambda}, \dots, P_{\text{rnd},d-1,1}, \dots, P_{\text{rnd},d-1,\lambda}) \in (\mathbb{F}_{2^\lambda}[X])^{(d-1)\lambda}$ such that $\mathbf{P}_{\text{rnd}}(0) \in \mathbb{F}_2^{(d-1)\lambda}$. Therefore, we propose a primitive named *batch line commitment* following the VOLEitH approach which is dedicated to commit (and later open evaluations of) these polynomials.

Gray code ϕ_{Gray} . The batch line commitment depends on a public one-to-one function $\phi : \{1, \dots, N\} \rightarrow \mathbb{F}_2^{1 \times \kappa}$. This function is used in the commitment procedure to commit polynomials while enabling the later opening of one evaluation on a point from $S := \{\phi(0), \dots, \phi(N-1)\}$. While the definition of ϕ has no importance in the correctness and the soundness of the scheme, it might impact the performance. In the SD-in-the-Head-2 signature scheme, we use the Gray code for ϕ , namely we use

$$\phi_{\text{Gray}} : i \in \{0, \dots, N-1\} \mapsto \text{bin}_\kappa(i) \oplus (\text{bin}_\kappa(i) \gg 1),$$

where

$$\begin{aligned} \text{bin}_\kappa(i) &:= (b_{\kappa-1}, \dots, b_0) \in \mathbb{F}_2^{1 \times \kappa}, \\ \text{bin}_\kappa(i) \gg 1 &:= (0, b_{\kappa-1}, \dots, b_1) \in \mathbb{F}_2^{1 \times \kappa}, \end{aligned}$$

with $i = \sum_{j=0}^{\kappa-1} b_j \cdot 2^j$. This code has the nice property that two consecutive values $\phi_{\text{Gray}}(i)$ and $\phi_{\text{Gray}}(i+1)$ differ on only one position.

In Algorithm 8 and Algorithm 10, ϕ_{Gray} is used to compute

$$\begin{aligned} \mathbf{r}_{\text{acc}} &\leftarrow \sum_{i=0}^{N-1} \mathbf{r}_{\text{rnd},i}, \\ \mathbf{r}_{\text{base}} &\leftarrow \sum_{i=0}^{N-1} \phi_{\text{Gray}}(i) \cdot \mathbf{r}_{\text{rnd},i}. \end{aligned}$$

Thanks to the structure of ϕ_{Gray} , we can compute $\mathbf{r}_{\text{acc}}$ and $\mathbf{r}_{\text{base}}$ with only $2N$ bit operations, while it would be in $O(N \cdot \kappa)$ if we would use a natural bit-representation for ϕ . Indeed, we can compute

$$\mathbf{r}_{\text{acc},j} \leftarrow \sum_{i=0}^j \mathbf{r}_{\text{rnd},i} \tag{4}$$

$$\mathbf{r}_{\text{base},j} \leftarrow \sum_{i=0}^j \mathbf{r}_{\text{acc},i} \cdot (\phi_{\text{Gray}}(i) \oplus \phi_{\text{Gray}}(i+1)) \tag{5}$$

for all $0 \leq j \leq N-1$, assuming $\phi_{\text{Gray}}(N) = 0$. Then, we can set $\mathbf{r}_{\text{acc}}$ and $\mathbf{r}_{\text{base}}$ respectively as

$\mathbf{r}_{\text{acc},N-1}$ and $\mathbf{r}_{\text{base},j}$. Indeed, it comes from

$$\begin{aligned}
 \mathbf{r}_{\text{base}} &= \sum_{j=0}^{N-1} \mathbf{r}_{\text{acc},j} \cdot (\phi_{\text{Gray}}(j) \oplus \phi_{\text{Gray}}(j+1)) \\
 &= \sum_{j=0}^{N-1} \left(\sum_{i=0}^j \mathbf{r}_{\text{rnd},i} \right) \cdot (\phi_{\text{Gray}}(j) \oplus \phi_{\text{Gray}}(j+1)) \\
 &= \sum_{i=0}^{N-1} \mathbf{r}_{\text{rnd},i} \cdot \sum_{j=i}^{N-1} (\phi_{\text{Gray}}(j) \oplus \phi_{\text{Gray}}(j+1)) \\
 &= \sum_{i=0}^{N-1} \mathbf{r}_{\text{rnd},i} \cdot \phi_{\text{Gray}}(i) .
 \end{aligned}$$

To sum up, we can compute $\mathbf{r}_{\text{acc}}$ using N bit operations using Equation (4), then we can compute $\mathbf{r}_{\text{base}}$ using just N additional bit operations using Equation (5) since $\phi_{\text{Gray}}(i) \oplus \phi_{\text{Gray}}(i+1)$ has only one non-zero bit at a public position.

The function ψ . The batch line commitment depends on a public one-to-one function $\psi : \mathbb{F}_2^{\tau \cdot \kappa} \rightarrow \mathbb{F}_{2^\lambda}$. In practice, we define ψ as

$$\psi : \mathbf{v} \in \mathbb{F}_2^{\tau \cdot \kappa} \mapsto \sum_{i=0}^{\tau \cdot \kappa - 1} v_i \cdot \xi^i,$$

where ξ is defined in Table 2.

Commitment routine. The commitment routine of the batched line commitment (BLC) scheme is described in Algorithm 8. After expanding τ sets of N seeds using the BAVC scheme, it computes the polynomials $\mathbf{P}^{(0)}, \dots, \mathbf{P}^{(\tau-1)}$ as

$$\mathbf{P}^{(e)} = \mathbf{r}_{\text{acc}}^{(e)} \cdot X + \mathbf{r}_{\text{base}}^{(e)}$$

for all $0 \leq e < \tau$, where

$$\begin{aligned}
 \mathbf{r}_{\text{acc}}^{(e)} &\leftarrow \sum_{i=1}^N \mathbf{r}_{\text{rnd},i}^{(e)} \in \mathbb{F}_2^{|\text{wit}| + (d-1)\lambda + (\lambda+B)} \\
 \mathbf{r}_{\text{base}}^{(e)} &\leftarrow - \sum_{i=1}^N \phi_{\text{Gray}}(i) \cdot \mathbf{r}_{\text{rnd},i}^{(e)} \in \mathbb{F}_{2^\kappa}^{|\text{wit}| + (d-1)\lambda + (\lambda+B)}
 \end{aligned}$$

with $\mathbf{r}_{\text{rnd},i}^{(e)} := \text{PRG.Init}(\text{seeds}[e][i])$. As explained in Section 2.1, the signer can reveal one evaluation of those polynomials among N , while keeping the others hidden. We want to merge those τ vector polynomials into a single polynomial for which the signer will be able to reveal one evaluation among N^τ . To proceed, we use the merging strategy of the VOLEitH framework.

However, since the merging strategy of the VOLEitH approach requires that $\mathbf{P}^{(0)}, \dots, \mathbf{P}^{(\tau-1)}$ encodes the same values (*i.e.* the leading terms should be the same), the routine computes an auxiliary value $\Delta \mathbf{r}^{(e)} := \mathbf{r}_{\text{acc}}^{(0)} - \mathbf{r}_{\text{acc}}^{(e)}$ for all $e > 0$ to define the polynomials $\mathbf{P}'^{(0)}, \dots, \mathbf{P}'^{(\tau-1)}$ as

$$\mathbf{P}'^{(e)} = \begin{cases} \mathbf{P}^{(e)} & \text{if } e = 0 \\ \mathbf{P}^{(e)} + (\Delta \mathbf{r}^{(e)}) \cdot X = \mathbf{r}_{\text{acc}}^{(0)} \cdot X + \mathbf{r}_{\text{base}}^{(e)} & \text{otherwise.} \end{cases}$$

From now, the signer has committed τ vector polynomials $\{\mathbf{P}'^{(e)}\}_{e < \tau}$ with the same leading term, but should convince the verifier that they really have the same leading term. To proceed, it runs the consistency check which consists in computing $\mathbf{P}'_{\alpha'}^{(e)} \in (\mathbb{F}_{2^\kappa})^{\lambda+B}$ as $[\mathbf{M} \mid \mathbf{I}_{\lambda+B}] \cdot \mathbf{P}'^{(e)}$ for all e . More precisely, it expands the consistency challenge $\mathbf{M}$ and computes

$$\alpha'_{\text{plain}} := [\mathbf{M} \mid \mathbf{I}_{\lambda+B}] \cdot \mathbf{r}_{\text{acc}}^{(0)} \in \mathbb{F}_2^{\lambda+B}$$

(leading term of all $\mathbf{P}'_{\alpha'}^{(e)}$'s) and

$$\alpha'_{\text{base}}^{(e)} := [\mathbf{M} \mid \mathbf{I}_{\lambda+B}] \cdot \mathbf{r}_{\text{base}}^{(e)} \in \mathbb{F}_{2^\kappa}^{\lambda+B}$$

(constant term of $\mathbf{P}'_{\alpha'}^{(e)}$) for all e .

After hashing the consistency check output $\{\mathbf{P}'_{\alpha'}^{(e)}\}_{e < \tau}$, the signer merges the τ vector polynomials $\mathbf{P}'^{(0)}, \dots, \mathbf{P}'^{(\tau-1)}$. The routine computes $\hat{\mathbf{P}} = \psi(\mathbf{P}'^{(0)}, \dots, \mathbf{P}'^{(\tau-1)})$ as:

$$\hat{\mathbf{P}}(X) = \mathbf{r}_{\text{acc}}^{(0)} \cdot X + \psi(\mathbf{r}_{\text{base}}^{(0)}, \dots, \mathbf{r}_{\text{base}}^{(1)}) \in (\mathbb{F}_{2^\lambda}[X])^{|\text{wit}| + (d-1) \cdot \lambda + (\lambda+B)}.$$

We then define $\hat{\mathbf{P}}_{\text{wit}}$ as the $|\text{wit}|$ first coordinates of $\hat{\mathbf{P}}$ and $\hat{\mathbf{P}}_{\text{rnd}}$ as the $\lambda \cdot (d-1)$ next coordinates. Then, the routine builds $\mathbf{P}_{\text{rnd}}$ as $X \cdot \hat{\mathbf{P}}_{\text{rnd}}(1/X)$ (*i.e.* it swaps the leading and the constant terms). For the witness polynomials, we need to enforce the constant term to be the input wit , so the routine computes Δ_{wit} as $\text{wit} - \hat{\mathbf{P}}(\infty)$ where $\hat{\mathbf{P}}(\infty)$ is the leading term of $\hat{\mathbf{P}}$ and build $\mathbf{P}_{\text{wit}}$ as $X \cdot \hat{\mathbf{P}}_{\text{wit}}(1/X) + \Delta_{\text{wit}}$.

The commitment routine outputs the committed degree-1 polynomials $\mathbf{P}_{\text{wit}}$ and $\mathbf{P}_{\text{rnd}}$, together with their BLC commitment h_{lines} , the BAVC key and the public auxiliary values.

Opening routine. The opening routine of the BLC scheme is described in Algorithm 9. It takes as input a BAVC opening key and a hash digest. Using the routine **ExpandEvaluationChallenge**, it expands a pseudo-random evaluation point Δ_{inv} defined as

$$\Delta_{\text{inv}} \leftarrow \psi([i^{*(0)}, \dots, i^{*(\tau-1)}]) ,$$

where $i^{*(e)}$'s are pseudo-random values from $\{0, \dots, N-1\}$. At the same time, it expands a random w_{pow} -bit grinding value v_{pow} . If the BAVC opening fails, the grinding value v_{pow} is not zero or if $\Delta_{\text{inv}} = 0$ (and hence fails to be invertible), the routine increments a counter and re-expands an other challenge until the three conditions are satisfied. It outputs the counter and the BAVC opening proof.

Recomputation routine. The recomputation routine of the BLC scheme is described in Algorithm 10. It first re-expands the evaluation challenge using **ExpandEvaluationChallenge** and checks $v_{\text{pow}} = 0$ and $\Delta_{\text{inv}} \neq 0$. After the check, it deduces the evaluation point $\Delta := (\Delta_{\text{inv}})^{-1}$. It gets all the opened seeds using **BAVC.Reconstruct**, *i.e.* all the seeds except those in $\{i^*[e]\}_{e < \tau}$. It computes $\mathbf{P}^{(0)}(\phi_{\text{Gray}}(i^{*(0)})), \dots, \mathbf{P}^{(\tau-1)}(\phi_{\text{Gray}}(i^{*(\tau-1)}))$ using the relation

$$\forall e, \mathbf{P}^{(e)}(\phi_{\text{Gray}}(i^{*(e)})) = \sum_{i=1, i \neq i^{*(e)}}^N \left(\phi_{\text{Gray}}(i^{*(e)}) - \phi_{\text{Gray}}(i) \right) \cdot \mathbf{r}_{\text{rnd},i}^{(e)} ,$$

with $\mathbf{r}_{\text{rnd},i}^{(e)} := \text{PRG.Init}(\text{seeds}[e][i])$. It then deduces $\mathbf{P}'^{(e)}(\phi_{\text{Gray}}(i^{*(e)}))$ for all e , using the relation

$$\forall e, \mathbf{P}'^{(e)}(\phi_{\text{Gray}}(i^{*(e)})) = \begin{cases} \mathbf{P}^{(e)}(\phi_{\text{Gray}}(i^{*(e)})) & \text{if } e = 0 \\ \mathbf{P}^{(e)}(\phi_{\text{Gray}}(i^{*(e)})) + (\Delta \mathbf{r}^{(e)}) \cdot \phi_{\text{Gray}}(i^{*(e)}) & \text{otherwise.} \end{cases}$$

Algorithm 8 BLC.Commit

Input: a salt $\text{salt} \in \{0, 1\}^\lambda$, a root seed $\text{rseed} \in \{0, 1\}^\lambda$, and a witness $\text{wit} \in \mathbb{F}_2^{|\text{wit}|}$

▷ Phase 1: Expand seeds.

1: $(\text{seeds}, h_{\text{com}}, \text{key}) \leftarrow \text{BAVC.Commit}(\text{salt}, \text{rseed})$

▷ Phase 2: Folding.

2: **for** e from 0 to $(\tau - 1)$ **do**

3: $\mathbf{r}_{\text{acc}}[e] = 0$

▷ $\mathbf{r}_{\text{acc}}[e] \in \mathbb{F}_2^{[|\text{wit}|+(d-1)\lambda+(\lambda+B)] \times 1}$

4: $\mathbf{r}_{\text{base}}[e] = 0$

▷ $\mathbf{r}_{\text{base}}[e] \in \mathbb{F}_2^{\kappa^{[|\text{wit}|+(d-1)\lambda+(\lambda+B)] \times 1}}$

5: **for** i from 0 to $(N - 1)$ **do**

6: $\text{prg} \leftarrow \text{PRG.Init}(\text{seeds}[e][i])$

7: $\mathbf{r}_{\text{rnd}} \leftarrow \text{PRG.SampleFieldElements}(\text{prg}, \mathbb{F}_2, |\text{wit}| + (d - 1)\lambda + (\lambda + B))$

8: $\mathbf{r}_{\text{acc}}[e] \mathrel{+}= \mathbf{r}_{\text{rnd}}$

9: $\mathbf{r}_{\text{base}}[e] \mathrel{+}= \mathbf{r}_{\text{rnd}} \cdot \phi_{\text{Gray}}(i)$

▷ $\phi_{\text{Gray}} : \{0, \dots, 2^\kappa - 1\} \rightarrow \mathbb{F}_2^{1 \times \kappa}$

10: **if** $e > 0$ **then**

11: $\mathbf{aux}[e] \leftarrow \mathbf{r}_{\text{acc}}[0] \oplus \mathbf{r}_{\text{acc}}[e]$

12: $\mathbf{u} \leftarrow \mathbf{r}_{\text{acc}}[0]$

▷ $\mathbf{u} \in \mathbb{F}_2^{[|\text{wit}|+(d-1)\lambda+(\lambda+B)] \times 1}$

13: $\mathbf{V} \leftarrow [\mathbf{r}_{\text{base}}[0], \dots, \mathbf{r}_{\text{base}}[\tau - 1]]$

▷ $\mathbf{V} \in \mathbb{F}_2^{[|\text{wit}|+(d-1)\lambda+(\lambda+B)] \times [\tau \cdot \kappa]}$

14: $h_{\text{aux}} = \text{Hash}_{\text{aux}}(h_{\text{com}}, \mathbf{aux}[1], \dots, \mathbf{aux}[\tau - 1])$

▷ Phase 3: Run consistency check.

15: $\mathbf{M} \leftarrow \text{ExpandConsistencyChallenge}(h_{\text{aux}})$

▷ $\mathbf{M} \in \mathbb{F}_2^{(\lambda+B) \times (|\text{wit}|+(d-1)\lambda)}$

16: $\alpha'_{\text{plain}} = [\mathbf{I}_{\lambda+B} \mid \mathbf{M}] \cdot \mathbf{u}$

▷ $\alpha'_{\text{plain}} \in \mathbb{F}_2^{[\lambda+B] \times 1}$

17: $\alpha'_{\text{base}} = [\mathbf{I}_{\lambda+B} \mid \mathbf{M}] \cdot \mathbf{V}$

▷ $\alpha'_{\text{base}} \in \mathbb{F}_2^{(\lambda+B) \times (\tau \cdot \kappa)}$

▷ Phase 4: Build lines.

18: $(\dots \parallel \mathbf{r}_{\text{wit}} \parallel \mathbf{r}_{\text{rnd}}) = \mathbf{u}$ where $|\mathbf{r}_{\text{wit}}| = |\text{wit}|$ and $|\mathbf{r}_{\text{rnd}}| = \lambda \cdot (d - 1)$

19: $(\mathbf{r}_{\text{base}, \text{wit}} \parallel \mathbf{r}_{\text{base}, \text{rnd}}) = (\psi(\mathbf{V}_{(\lambda+B)+1}) \parallel \dots \parallel \psi(\mathbf{V}_{(\lambda+B)+|\text{wit}|+(d-1)\lambda}))$

20: ▷ $\psi : \mathbb{F}_2^{\tau \cdot \kappa} \rightarrow \mathbb{F}_2^{\lambda}$, $\mathbf{V}_i$'s are rows of $\mathbf{V}$

21: $\Delta \text{wit} = \text{wit} \oplus \mathbf{r}_{\text{wit}}$

22: $P_{\text{wit}}(X) \leftarrow \text{wit} + \mathbf{r}_{\text{base}, \text{wit}} \cdot X$

23: $P_{\text{rnd}}(X) \leftarrow \mathbf{r}_{\text{rnd}} + \mathbf{r}_{\text{base}, \text{rnd}} \cdot X$

24: $h_{\text{lines}} = \text{Hash}_{\text{lines}}(h_{\text{aux}}, \alpha'_{\text{plain}}, \alpha'_{\text{base}}, \Delta \text{wit})$

25: $\mathbf{aux}' \leftarrow (\mathbf{aux} \parallel \alpha'_{\text{plain}} \parallel \Delta \text{wit})$

26: **return** $(P_{\text{wit}}, P_{\text{rnd}}, h_{\text{lines}}, \text{key}, \mathbf{aux}')$

Now, the verifier needs to check the consistency test. After expanding the consistency challenge $\mathbf{M}$ using **ExpandConsistencyChallenge**, it computes $\mathbf{P}_{\alpha'}^{(e)}(\phi_{\text{Gray}}(i^{*(e)}))$ as $[\mathbf{M} \mid \mathbf{I}_{\lambda+B}] \mathbf{P}'^{(e)}(\phi_{\text{Gray}}(i^{*(e)}))$ and deduces the constant term $\alpha'_{\text{base}}^{(e)}$ of $\mathbf{P}_{\alpha'}^{(e)}$ as $\alpha'_{\text{base}}^{(e)} := \mathbf{P}_{\alpha'}^{(e)}(\phi_{\text{Gray}}(i^{*(e)})) - \alpha'_{\text{plain}}^{(e)} \cdot \phi_{\text{Gray}}(i^{*(e)})$.

After hashing the consistency check output $\{\mathbf{P}_{\alpha'}^{(e)}\}_{e < \tau}$, the verifier computes $\hat{\mathbf{P}}(\Delta_{\text{inv}})$ by

Algorithm 9 **BLC.OpenRandomEvaluation****Input:** a BAVC opening key key and a hash digest $h_{\text{piop}} \in \{0, 1\}^{2\lambda}$

```

1:  $\text{ctr} \leftarrow 0$   $\triangleright$  32-bit counter
2: retry:
3:  $(\{i^*[e]\}_{e < \tau}, v_{\text{pow}}) \leftarrow \text{ExpandEvaluationChallenge}(h_{\text{piop}}, \text{ctr})$ 
4:  $\Delta_{\text{inv}} \leftarrow \psi([i^*[0], \dots, i^*[\tau - 1]])$   $\triangleright \Delta_{\text{inv}} \in \mathbb{F}_{2^\lambda}$ 
5:  $\pi_{\text{BAVC}} \leftarrow \text{BAVC.Open}(\text{key}, \{i^*[e]\}_{e < \tau})$ 
6: if  $\pi_{\text{BAVC}} = \perp$  or  $v_{\text{pow}} \neq 0$  or  $\Delta_{\text{inv}} = 0$  then
7:    $\text{ctr} \leftarrow \text{ctr} + 1$ 
8:   goto retry
9:  $\text{pdecom} = (\text{ctr}, \pi_{\text{BAVC}})$ 
10: return  $\text{pdecom}$ 

```

merging the τ evaluations $\mathbf{P}'^{(0)}(\phi_{\text{Gray}}(i^{*(0)})), \dots, \mathbf{P}'^{(\tau-1)}(\phi_{\text{Gray}}(i^{*(\tau-1)}))$ as:

$$\hat{\mathbf{P}}(\Delta_{\text{inv}}) := \psi \left(\mathbf{P}'^{(0)}(\phi_{\text{Gray}}(i^{*(0)})), \dots, \mathbf{P}'^{(\tau-1)}(\phi_{\text{Gray}}(i^{*(\tau-1)})) \right) \in \mathbb{F}_{2^\lambda}^{|\text{wit}| + (d-1)\lambda + (\lambda+B)}.$$

They get $\hat{\mathbf{P}}_{\text{wit}}(\Delta_{\text{inv}})$ as the $|\text{wit}|$ first coordinates of $\hat{\mathbf{P}}(\Delta_{\text{inv}})$ and $\hat{\mathbf{P}}_{\text{rnd}}(\Delta_{\text{inv}})$ as the $\lambda \cdot (d-1)$ next coordinates. Then the routine computes $\mathbf{p}_{\text{rnd}} := \mathbf{P}_{\text{rnd}}(\Delta)$ as $\Delta \cdot \hat{\mathbf{P}}_{\text{rnd}}(\Delta_{\text{inv}})$ and $\mathbf{p}_{\text{wit}} := \mathbf{P}_{\text{wit}}(\Delta)$ as $\Delta \cdot \hat{\mathbf{P}}_{\text{wit}}(\Delta_{\text{inv}}) + \Delta_{\text{wit}}$.

The reconstruction routine outputs the evaluations $\mathbf{p}_{\text{wit}}$ and $\mathbf{p}_{\text{rnd}}$, together with the evaluation point Δ and the BLC commitment h_{lines} .

Algorithm 10 `BLC.RecomputeEvaluation`

Input: a BLC auxiliary value aux' , a BLC opening pdecom , a salt $\text{salt} \in \{0, 1\}^\lambda$ and a hash digest $h_{\text{piop}} \in \{0, 1\}^{2\lambda}$

▷ Phase 1: Get evaluation point

- 1: $(\text{ctr}, \pi_{\text{BAVC}}) = \text{pdecom}$
- 2: $(\text{aux} \parallel \alpha'_{\text{plain}} \parallel \Delta_{\text{wit}}) \leftarrow \text{aux}'$
- 3: $(\{i^*[e]\}_{e < \tau}, v_{\text{pow}}) \leftarrow \text{ExpandEvaluationChallenge}(h_{\text{piop}}, \text{ctr})$
- 4: $\Delta_{\text{inv}} \leftarrow \psi([i^*[0], \dots, i^*[\tau - 1]])$ ▷ $\Delta_{\text{inv}} \in \mathbb{F}_{2^\lambda}$
- 5: **if** $v_{\text{pow}} \neq 0$ **or** $\Delta_{\text{inv}} = 0$ **then**
- 6: **return** $(\perp, \perp, \perp, \perp)$
- 7: $\Delta \leftarrow (\Delta_{\text{inv}})^{-1}$

▷ Phase 2: Expand seeds.

- 8: $h_{\text{com}}, \text{seeds} \leftarrow \text{BAVC.Reconstruct}(\{i^*[e]\}_{e < \tau}, \pi_{\text{BAVC}}, \text{salt})$

▷ Phase 3: Folding.

- 9: **for** e from 0 to $(\tau - 1)$ **do**
- 10: $r_{\text{eval}}[e] = 0$ ▷ $r_{\text{eval}}[e] \in \mathbb{F}_{2^\kappa}^{[|\text{wit}| + (d-1)\lambda + (\lambda+B)] \times 1}$
- 11: **for** $i \in \{0, \dots, (N - 1)\} \setminus i^*[e]$ **do**
- 12: $\text{prg} \leftarrow \text{PRG.Init}(\text{seed}[e][i])$
- 13: $r_{\text{rnd}} \leftarrow \text{PRG.SampleFieldElements}(\text{prg}, \mathbb{F}_2, |\text{wit}| + (d - 1)\lambda + (\lambda + B))$
- 14: $r_{\text{share}}[e] \mathrel{+}= r_{\text{rnd}} \cdot (\phi_{\text{Gray}}(i^*[e]) - \phi_{\text{Gray}}(i))$ ▷ $\phi_{\text{Gray}} : \{0, \dots, 2^\kappa - 1\} \rightarrow \mathbb{F}_2^{1 \times [\kappa]}$
- 15: **if** $e > 0$ **then**
- 16: $r_{\text{share}}[e] \mathrel{+}= \text{aux}[e] \cdot \phi_{\mu_e}(i^*[e])$
- 17: $Q \leftarrow [r_{\text{share}}[0], \dots, r_{\text{share}}[\tau - 1]]$ ▷ $Q \in \mathbb{F}_2^{[|\text{wit}| + (d-1)\lambda + (\lambda+B)] \times [\tau \cdot \kappa]}$
- 18: $h_{\text{aux}} = \text{Hash}_{\text{aux}}(h_{\text{com}}, \text{aux}[1], \dots, \text{aux}[\tau - 1])$

▷ Phase 4: Run consistency check.

- 19: $M \leftarrow \text{ExpandConsistencyChallenge}(h_{\text{aux}})$ ▷ $M \in \mathbb{F}_2^{(\lambda+B) \times (|\text{wit}| + (d-1)\lambda)}$
- 20: $\alpha'_{\text{eval}} \leftarrow [I_{\lambda+B} \mid M] \cdot Q$ ▷ $\alpha'_{\text{eval}} \in \mathbb{F}_2^{[\lambda+B] \times [\tau \cdot \kappa]}$
- 21: $\alpha'_{\text{base}} \leftarrow \alpha'_{\text{eval}} - \alpha'_{\text{plain}} \cdot \Delta_{\text{inv}}$ ▷ $\alpha'_{\text{base}} \in \mathbb{F}_2^{[\lambda+B] \times [\tau \cdot \kappa]}$

▷ Phase 5: Build line evaluation.

- 22: $(r_{\text{eval}, \text{wit}} \parallel r_{\text{eval}, \text{rnd}}) = (\psi(Q_{\lambda+B+1}) \parallel \dots \parallel \psi(Q_{(\lambda+B)+|\text{wit}|+(d-1)\lambda}))$
- 23: ▷ $\psi : \mathbb{F}_2^{\tau \cdot \kappa} \rightarrow \mathbb{F}_{2^\lambda}$, Q_i 's are rows of Q
- 24: $p_{\text{wit}} \leftarrow \Delta_{\text{wit}} \oplus (\Delta \cdot r_{\text{eval}, \text{wit}})$
- 25: $p_{\text{rnd}} \leftarrow \Delta \cdot r_{\text{eval}, \text{rnd}}$
- 26: $h_{\text{lines}} = \text{Hash}_{\text{lines}}(h_{\text{aux}}, \alpha'_{\text{plain}}, \alpha'_{\text{base}})$
- 27: **return** $(\Delta, p_{\text{wit}}, p_{\text{rnd}}, h_{\text{lines}})$

3.2.6 PIOP protocol

As explained in [Section 2.1](#) and [Section 3.1](#), the PIOP protocol aims to compute the degree- d polynomial P_α such that

$$P_\alpha(X) = P_0(X) \cdot X + \sum_{j=1}^m \gamma_j \cdot f_j(P_1(X), \dots, P_{|\text{wit}|}(X)).$$

where

- $P_1, \dots, P_{|\text{wit}|}$ are the witness polynomials, *i.e.* $\mathbf{P}_{\text{wit}} := (P_1, \dots, P_{|\text{wit}|})$;
- P_0 is the degree- $(d-1)$ masking polynomial built as

$$P_0(X) := \sum_{i=0}^{d-2} \left(\sum_{j=0}^{\lambda-1} \xi^j \cdot P_{\text{rnd},i,j}(X) \right) \cdot X^i$$

with $\mathbf{P}_{\text{rnd}} := (P_{\text{rnd},0,0}, \dots, P_{\text{rnd},0,\lambda-1}, \dots, P_{\text{rnd},d-2,0}, \dots, P_{\text{rnd},d-2,\lambda-1})$ and $(1, \xi, \xi^2 \dots)$ is a $\mathbb{F}_2$ -basis of $\mathbb{F}_{2^\lambda}$;

- $\{f_j\}_j$ are the degree- d polynomial constraints that the SD-in-the-Head-2 witness wit should satisfy (see [Section 2.2](#)).

We have two types of polynomial constraints:

1. The first constraints consist of checking that the elementary vectors used to build the chunks of $\mathbf{x}$ by tensor products have exactly one non-zero coordinate. Since the last coordinates of those vectors is discarded from the witness (and then recovered as 1 minus the sum of the other coordinates), we actually need to check that there is at most one non-zero coordinate.
2. The other constraints consist of checking that the vector $\mathbf{x}$ obtained by concatenating the tensor products of the elementary vectors satisfies the linear relation $\mathbf{y} = \mathbf{H}\mathbf{x}$.

Parsing the input polynomials. The PIOP protocol takes the input vector polynomials $\mathbf{P}_{\text{wit}}$ and $\mathbf{P}_{\text{rnd}}$. The vector polynomial $\mathbf{P}_{\text{wit}} := (P_1, \dots, P_{|\text{wit}|})$ encodes the RSD witness wit , a bit-string made of the decomposition of the RSD solution $\mathbf{x}$ as elementary vectors (input of tensor products). The RSD witness wit can be parsed as bits $\{w_{i,j,k}\}_{i,j,k}$, where $w_{i,j,k}$ is the k^{th} coordinates of the j^{th} elementary vector in the tensor product of the i^{th} chunk of the RSD solution $\mathbf{x}$. By parsing $\mathbf{P}_{\text{wit}}$ in the same way, we can get the degree-1 polynomials $\{P_{w_{i,j,k}}\}_{i,j,k}$ that encode the bits $\{w_{i,j,k}\}_{i,j,k}$.

The vector $\mathbf{P}_{\text{rnd}} := (P_{\text{rnd},1,1}, \dots, P_{\text{rnd},1,\lambda}, \dots, P_{\text{rnd},d-1,1}, \dots, P_{\text{rnd},d-1,\lambda})$ contains $\lambda \cdot (d-1)$ degree-1 polynomials with leading term from $\mathbb{F}_{2^\lambda}$ and constant term from $\mathbb{F}_2$. These polynomials are used to build a degree- $(d-1)$ random polynomial $P_0 \in \mathbb{F}_{2^\lambda}[X]$ as:

$$P_0(X) := \sum_{i=0}^{d-2} \left(\sum_{j=0}^{\lambda-1} \xi^j \cdot P_{\text{rnd},i,j}(X) \right) \cdot X^i.$$

We describe in [Algorithm 11](#) the routine that takes as inputs the vector polynomials $\mathbf{P}_{\text{wit}}$ and $\mathbf{P}_{\text{rnd}}$ and outputs the set of degree-1 polynomials $\{P_{w_{i,j,k}}\}_{i,j,k}$ that encode the bits $\{w_{i,j,k}\}_{i,j,k}$

of the decomposition of the RSD solution and the degree- $(d - 1)$ polynomial P_0 . We then describe in [Algorithm 12](#) the routine that takes as inputs the vectors $p_{\text{wit}} := \mathbf{P}_{\text{wit}}(\Delta) \in \mathbb{F}_{2^\lambda}^{|\text{wit}|}$ and $p_{\text{rnd}} := \mathbf{P}_{\text{rnd}}(\Delta) \in \mathbb{F}_{2^\lambda}^{|\text{rnd}|}$ and outputs the set $\{P_{w_{i,j,k}}(\Delta)\}_{i,j,k}$ and the value $P_0(\Delta)$, for some field element $\Delta \in \mathbb{F}_{2^\lambda}$.

Algorithm 11 [PIOP.Prover.Format](#)

Input: two degree-1 vector polynomials $\mathbf{P}_{\text{wit}}$ and $\mathbf{P}_{\text{rnd}}$.

```

1: ind  $\leftarrow$  0
2: for  $i$  from 0 to  $w - 1$  do
3:   for  $j$  from 0 to  $d - 1$  do
4:     for  $k$  from 0 to  $\mu_j - 2$  do
5:        $P_{w_{i,j,k}} \leftarrow (P_{\text{wit}})_{\text{ind}}$   $\triangleright$  Degree-1 polynomial
6:       ind  $\leftarrow$  ind + 1
7:  $P_0(X) \leftarrow \sum_{i=0}^{d-2} \left( \sum_{j=0}^{\lambda-1} \xi^j \cdot (P_{\text{rnd}}(X))_{i\lambda+j} \right) \cdot X^i$   $\triangleright$  Degree- $(d - 1)$  polynomial
8: return  $\{P_{w_{i,j,k}}\}_{i,j,k}, P_0$ 

```

Algorithm 12 [PIOP.Verifier.Format](#)

Input: a field element $\Delta \in \mathbb{F}_{2^\lambda}$, two vectors p_{wit} and p_{rnd} $\triangleright p_{\text{wit}} := \mathbf{P}_{\text{wit}}(\Delta), p_{\text{rnd}} := \mathbf{P}_{\text{rnd}}(\Delta)$

```

1: ind  $\leftarrow$  0
2: for  $i$  from 0 to  $w - 1$  do
3:   for  $j$  from 0 to  $d - 1$  do
4:     for  $k$  from 0 to  $\mu_j - 2$  do
5:        $p_{w_{i,j,k}} \leftarrow (p_{\text{wit}})_{\text{ind}}$ 
6:       ind  $\leftarrow$  ind + 1
7:  $p_0 \leftarrow \sum_{i=0}^{d-2} \left( \sum_{j=0}^{\lambda-1} \xi^j \cdot (p_{\text{rnd}})_{i\lambda+j} \right) \cdot \Delta^i$ 
8: return  $\{p_{w_{i,j,k}}\}_{i,j,k}, p_0$ 

```

Checking elementary vectors. To check that a binary vector $(w_{i,j,0}, \dots, w_{i,j,\mu_j-2}) \in \mathbb{F}_2^{\mu_j-1}$ has at most one non-zero coordinate, we check that for all $k \neq k'$, we have $w_{i,j,k} \cdot w_{i,j,k'} = 0$. However, to avoid performing $O(\mu_j^2)$ multiplications, we instead check that the polynomial $D_{i,j} \in \mathbb{F}_2[Y]$ defined as:

$$D_{i,j}(Y) := \left(\sum_{k=0}^{\mu_j-2} Y^k \cdot w_{i,j,k} \right) \cdot \left(\sum_{k=0}^{\mu_j-3} Y^{(\mu_j-1)k} \cdot w_{i,j,k} \right) - \left(\sum_{k=0}^{\mu_j-3} Y^{\mu_j \cdot k} \cdot w_{i,j,k} \right)$$

equals $0 \in \mathbb{F}_2[Y]$ for all $1 \leq i \leq w$ and $1 \leq j \leq d$. For our considered parameters, the degree of these polynomials always satisfies

$$\deg(D_{i,j}) = (\mu_j - 3) \cdot \mu_j + 1 < 32 .$$

Moreover, we always have $d \leq 4$. Those two upper bounds imply:

$$\forall 1 \leq j \leq d, D_{i,j}(Y) = 0 \in \mathbb{F}_2[Y] \Leftrightarrow \sum_{j=1}^d \xi^{32(j-1)} \cdot D_{i,j}(\xi) = 0 \in \mathbb{F}_{2^\lambda} .$$

The check of the elementary vectors hence reduces to checking the right-hand side of the above equivalence.

Looking ahead, these relations will be batched using random coefficients for the definition of the polynomial P_α . Let $\mathbf{v}' = (v'_1, \dots, v'_w) \in \mathbb{F}_{2^\lambda}^w$ the vector with coordinates defined as

$$v'_i = \sum_{j=1}^d \xi^{32(j-1)} \cdot D_{i,j}(\xi) , \quad (6)$$

and let $\gamma' \in \mathbb{F}_{2^\lambda}^w$ be the challenge from the verifier. Checking the elementary vectors shall consist in checking $\gamma'^\top \cdot \mathbf{v}' = 0$ which implies $\mathbf{v}' = 0$ with overwhelming probability over the randomness of γ' .

The routine **PIOP.Prover.UnitaryCheck** performs the computation of $D_{i,j}(\xi)$ from a prover standpoint, where each witness bit $w_{i,j,k}$ is encoded through a degree-1 polynomial $P_{w_{i,j,k}}(X)$. The routine **PIOP.Verifier.UnitaryCheck** performs the computation of $D_{i,j}(\xi)$ from a verifier standpoint, where each witness bit $w_{i,j,k}$ is encoded through a polynomial evaluation $p_{w_{i,j,k}} := P_{w_{i,j,k}}(\Delta)$.

Algorithm 13 **PIOP.Prover.UnitaryCheck**

Input: arity μ_j , degree-1 polynomials $P_{w_{i,j,0}}(X), \dots, P_{w_{i,j,\mu_j-2}}(X) \in \mathbb{F}_{2^\lambda}[X]$ for which the constant terms are $w_{i,j,0}, \dots, w_{i,j,\mu_j-2} \in \mathbb{F}_2$.

- 1: $\text{sum}_1 \leftarrow \sum_{k=0}^{\mu_j-2} \xi^k \cdot P_{w_{i,j,k}}(X)$
 - 2: $\text{sum}_2 \leftarrow \sum_{k=0}^{\mu_j-3} \xi^{(\mu_j-1)k} \cdot P_{w_{i,j,k}}(X)$
 - 3: $\text{sum}_3 \leftarrow \sum_{k=0}^{\mu_j-3} \xi^{\mu_j \cdot k} \cdot P_{w_{i,j,k}}(X)$
 - 4: **return** $\text{sum}_1 \cdot \text{sum}_2 - \text{sum}_3$ ▷ Degree-1 polynomial (since the witness is well-built)
-

Algorithm 14 **PIOP.Verifier.UnitaryCheck**

Input: arity μ_j , evaluations $p_{w_{i,j,0}}, \dots, p_{w_{i,j,\mu_j-2}} \in \mathbb{F}_{2^\lambda}$ ▷ $\forall k, p_{w_{i,j,k}} := P_{w_{i,j,k}}(\Delta)$

- 1: $\text{sum}_1 \leftarrow \sum_{k=0}^{\mu_j-2} \xi^k \cdot p_{w_{i,j,k}}$
 - 2: $\text{sum}_2 \leftarrow \sum_{k=0}^{\mu_j-3} \xi^{(\mu_j-1)k} \cdot p_{w_{i,j,k}}$
 - 3: $\text{sum}_3 \leftarrow \sum_{k=0}^{\mu_j-3} \xi^{\mu_j \cdot k} \cdot p_{w_{i,j,k}}$
 - 4: **return** $\text{sum}_1 \cdot \text{sum}_2 - \text{sum}_3$
-

Checking RSD linear constraints. We need to check the relation $\mathbf{H}\mathbf{x} = \mathbf{y}$ where $\mathbf{x} = (e_0 \parallel \dots \parallel e_{w-1})$ is built as the concatenation of the elementary vectors e_i resulting from the tensor products. Namely, we need to check

$$\mathbf{v} := \sum_{i=0}^{w-1} \sum_{j=0}^{m-1} (e_i)_j \cdot \mathbf{h}_{i \cdot m + j} - \mathbf{y} = (0, \dots, 0) \in \mathbb{F}_2^{n-k} ,$$

where $[\mathbf{h}_0 \mid \mathbf{h}_1 \mid \dots \mid \mathbf{h}_{n-1}] = \mathbf{H}$.

Looking ahead, the coordinates of this vector (encoded as polynomials) will be batched using random coefficients in the computation of P_α . To make this batching more efficient, we embed

blocks from $\mathbb{F}_2^\lambda$ into elements of $\mathbb{F}_{2^\lambda}$. Let ϕ be the linear $\mathbb{F}_2$ -linear field-embedding isomorphism:

$$\phi : (v_1, \dots, v_\lambda) \in \mathbb{F}_2^\lambda \mapsto \sum_{i=1}^{\lambda} v_i \cdot \xi^{i-1} ,$$

and let Φ be its block-wise variant:

$$\Phi : \mathbf{v} \in \mathbb{F}_2^{n-k} \mapsto (\phi(v_1, \dots, v_\lambda), \dots, \phi(v_{n-k}, 0, \dots, 0)) \in \mathbb{F}_{2^\lambda}^{\lceil \frac{n-k}{\lambda} \rceil} , \quad (7)$$

where the last block is padded with $\lambda \cdot \lceil \frac{n-k}{\lambda} \rceil - (n-k)$ zeros. We have

$$\mathbf{v} = 0 \in \mathbb{F}_2^{n-k} \Leftrightarrow \Phi(\mathbf{v}) = 0 \in \mathbb{F}_{2^\lambda}^{\lceil \frac{n-k}{\lambda} \rceil} .$$

Now let $\gamma \in \mathbb{F}_{2^\lambda}^{\lceil \frac{n-k}{\lambda} \rceil}$ be the challenge from the verifier. Checking the RSD linear constraints shall consist in checking $\gamma^\top \cdot \Phi(\mathbf{v}) = 0$ which implies $\Phi(\mathbf{v}) = 0$ (and hence $\mathbf{v} = 0$) with overwhelming probability over the randomness of γ . Then we observe that $\gamma^\top \cdot \Phi(\mathbf{v})$ can be written as:

$$\gamma^\top \cdot \Phi(\mathbf{v}) = \sum_{i=0}^{w-1} \sum_{j=0}^{m-1} (e_i)_j \cdot h_{i \cdot m + j}^{[\gamma]} - y^{[\gamma]} , \quad (8)$$

where

$$h_j^{[\gamma]} := \gamma^\top \cdot \Phi(\mathbf{h}_j) \quad \forall 0 \leq j \leq n-1 \quad \text{and} \quad y_j^{[\gamma]} := \gamma^\top \cdot \Phi(\mathbf{y}) . \quad (9)$$

Mux tree for efficient evaluation. To compute $\gamma^\top \cdot \Phi(\mathbf{v})$, either as polynomial encoding on the prover side (i.e., in the computation of P_α) or as evaluation encoding on the verifier side (i.e., in the computation of $P_\alpha(\Delta)$), a natural option would be to expand each elementary vector e_i from the witness bits $\{w_{i,j,k}\}_{j,k}$ and evaluate Equation 8. However, this approach would involve many field multiplications. We instead rely on a multiplexer tree (or “mux tree” for short).

For a vector $\mathbf{b} = (b_0, \dots, b_{\mu_j-2}) \in \{0,1\}^{\mu_j-1}$ such that $w_H(\mathbf{b}) \leq 1$, and a vector $\mathbf{u} = (u_0, \dots, u_{\mu_j}) \in \mathbb{F}_{2^\lambda}^{\mu_j}$, we define the μ_j -to-1 multiplexer gate as follows:

$$\text{Mux}_{\mu_j} : (\mathbf{b}, \mathbf{u}) \mapsto \begin{cases} u_0 & \text{if } \mathbf{b} = 0 \\ u_1 & \text{if } b_0 = 1 \\ \vdots & \\ u_{\mu_j-1} & \text{if } b_{\mu_j-2} = 1 \end{cases}$$

This gate can be efficiently implemented using $\mu_j - 1$ multiplications:

$$\text{Mux}_{\mu_j}(\mathbf{b}, \mathbf{u}) = u_0 + \sum_{j=1}^{\mu_j-1} b_{j-1} \cdot (u_j - u_0).$$

As explained in Section 2.2, the elementary vectors e_i 's composing the RSD solution $\mathbf{x} = (e_0 \parallel \dots \parallel e_{m-1})$ are defined as:

$$e_i \leftarrow \begin{pmatrix} b_{i,1,1} \\ b_{i,1,2} \\ \vdots \\ b_{i,1,\mu_1-1} \\ 1 - \sum_k b_{i,1,k} \end{pmatrix} \otimes \begin{pmatrix} b_{i,2,1} \\ b_{i,2,2} \\ \vdots \\ b_{i,2,\mu_2-1} \\ 1 - \sum_k b_{i,2,k} \end{pmatrix} \otimes \dots \otimes \begin{pmatrix} b_{i,d,1} \\ b_{i,d,2} \\ \vdots \\ b_{i,d,\mu_d-1} \\ 1 - \sum_k b_{i,d,k} \end{pmatrix} \in \mathbb{F}^{n/w} .$$

For all i , we can observe that computing $\sum_{j=0}^m (e_i)_j \cdot h_{i \cdot m+j}^{[\gamma]}$ is equivalent to computing the root $\text{node}_{d,0}$ of a mux tree with arities $(\mu_1, \dots, \mu_d)$. Formally, for all $0 \leq j \leq d$ and $0 \leq \ell < \mu_{j+1} \cdot \dots \cdot \mu_d$, let

$$\text{node}_{j,\ell} = \begin{cases} h_{i \cdot m+\ell}^{[\gamma]} & \text{if } j = 0 \text{ (the tree leaves)} \\ \text{Mux}_{\mu_j} \left(\mathbf{b}_{i,j}, (\text{node}_{j-1,\mu_j \cdot h}, \dots, \text{node}_{j-1,\mu_j \cdot (\ell+1)-1}) \right) & \text{if } j > 0 \text{ (the tree nodes)} \end{cases}$$

where $\mathbf{b}_{i,j} := (b_{i,j,1}, \dots, b_{i,j,\mu_j-1})$. We denote $\text{MuxTree}((h_{i \cdot m}^{[\gamma]}, \dots, h_{(i+1) \cdot m-1}^{[\gamma]}), \{\mathbf{b}_{i,j}\}_j)$ the root of the mux tree. Equation 8 now rewrites as:

$$\gamma^\top \cdot \Phi(\mathbf{v}) = \sum_{i=0}^{w-1} \text{MuxTree}((h_{i \cdot m}^{[\gamma]}, \dots, h_{(i+1) \cdot m-1}^{[\gamma]}), \{\mathbf{b}_{i,j}\}_j) .$$

We describe in Algorithm 15 the computation of the mux-tree root from the prover standpoint, where each witness bit $w_{i,j,k}$ is encoded through a degree-1 polynomial $P_{w_{i,j,k}}(X)$. We then describe in Algorithm 16 the computation of the mux-tree root from the verifier standpoint, where each witness bit $w_{i,j,k}$ is encoded through a polynomial evaluation.

Algorithm 15 `PIOP.Prover.MuxTree`

Input: coefficients $(h_{i \cdot m}^{[\gamma]}, \dots, h_{(i+1) \cdot m-1}^{[\gamma]}) \in \mathbb{F}_{2^\lambda}^m$,
degree-1 polynomials $\{(P_{w_{i,j,0}}(X), \dots, P_{w_{i,j,\mu_j-2}}(X))\}_j$ of $\mathbb{F}_{2^\lambda}[X]$

- 1: $(\text{node}_{0,0}, \dots, \text{node}_{0,m-1}) \leftarrow (h_{i \cdot m}^{[\gamma]}, \dots, h_{(i+1) \cdot m-1}^{[\gamma]})$ ▷ degree-0 polynomials
- 2: **for** j from 1 to d **do**
- 3: **for** h from 0 to $\mu_{j+1} \cdot \dots \cdot \mu_d - 1$ **do**
- 4: $\text{node}_{j,\ell} \leftarrow \text{node}_{j-1,\mu_j \cdot h} + \sum_{k=0}^{\mu_j-2} P_{w_{i,j,k}}(X) \cdot (\text{node}_{j-1,\mu_j \cdot h+(1+k)} - \text{node}_{j-1,\mu_j \cdot h})$ ▷ degree- j polynomials
- 5: **return** $\text{node}_{d,0}$ ▷ degree- d polynomial

Algorithm 16 `PIOP.Verifier.MuxTree`

Input: coefficients $(h_{i \cdot m}^{[\gamma]}, \dots, h_{(i+1) \cdot m-1}^{[\gamma]}) \in \mathbb{F}_{2^\lambda}^m$,
evaluations $\{(p_{w_{i,j,0}}, \dots, p_{w_{i,j,\mu_j-2}})\}_j \in \mathbb{F}_{2^\lambda}^{\mu_1-1} \times \dots \times \mathbb{F}_{2^\lambda}^{\mu_d-1}$ ▷ $p_{w_{i,j,k}} := P_{w_{i,j,k}}(\Delta)$

- 1: $(\text{node}_{0,0}, \dots, \text{node}_{0,m-1}) \leftarrow (h_{i \cdot m}^{[\gamma]}, \dots, h_{(i+1) \cdot m-1}^{[\gamma]})$
- 2: **for** j from 1 to d **do**
- 3: **for** h from 0 to $\mu_{j+1} \cdot \dots \cdot \mu_d - 1$ **do**
- 4: $\text{node}_{j,h} \leftarrow \text{node}_{j-1,\mu_j \cdot h} + \sum_{k=0}^{\mu_j-2} p_{w_{i,j,k}} \cdot (\text{node}_{j-1,\mu_j \cdot h+(1+k)} - \text{node}_{j-1,\mu_j \cdot h})$
- 5: **return** $\text{node}_{d,0}$

PIOP Protocol. We describe in Algorithm 17 the prover's computation in the PIOP protocol, namely how the prover compute the polynomial P_α from the masking polynomial P_0 and the witness $\mathbf{P}_{\text{wit}} := (P_1, \dots, P_{|\text{wit}|})$. The first step consists in formatting the input polynomials

using the routine **PIOP.Prover.Format**: it parses the vector polynomial $\mathbf{P}_{\text{wit}}$ as $\{P_{w_{i,j,k}}\}_{i,j,k}$ and builds the degree- $(d-1)$ masking polynomial P_0 from the degree-1 vector polynomial P_{rnd} . Next, the verifier challenge (γ', γ) , i.e., the randomness for batching the relations, is generated by hashing the BLC commitment h_{lines} . Then the batched field-embedded elements $\{h_j^{[\gamma]}\}$ and $y^{[\gamma]}$ are computed (see Equation 9). Finally, the polynomial P_α is computed as:

$$\begin{aligned}
 P_\alpha(X) &= P_0(X) \cdot X + \sum_j \gamma_j \cdot f_j(P_1(X), \dots, P_{|\text{wit}|}(X)). \\
 &= P_0(X) \cdot X \\
 &\quad + \underbrace{\sum_{i=0}^{w-1} \text{PIOP.Prover.MuxTree}\left((h_{i \cdot m}^{[\gamma]}, \dots, h_{(i+1) \cdot m-1}^{[\gamma]}), \{P_{w_{i,j,k}}(X)\}_{j,k}\right)}_{\text{polynomial encoding } \gamma^\top \cdot \Phi(\mathbf{v})} \\
 &\quad + \underbrace{\sum_{i=0}^{w-1} \gamma'_i \cdot \sum_{j=0}^{d-1} \xi^{32(j-1)} \cdot \text{PIOP.Prover.UnitaryCheck}(\mu_j, \{P_{w_{i,j,k}}(X)\}_k)}_{\text{polynomial encoding } \gamma'^\top \cdot \mathbf{v}'}
 \end{aligned}$$

Algorithm 17 **PIOP.Prover.Run**

Input: two vector polynomials $P_{\text{wit}} \in (\mathbb{F}_{2^\lambda}[X])^{|\text{wit}|}$ and $P_{\text{rnd}} \in (\mathbb{F}_{2^\lambda}[X])^{|\text{rnd}|}$, a hash digest $h_{\text{lines}} \in \{0, 1\}^{2^\lambda}$ and the regular syndrome instance $(\mathbf{H}', \mathbf{y})$.

Output: a degree- d polynomial P_α of $\mathbb{F}_{2^\lambda}[X]$

- 1: $(\{P_{w_{i,j,k}}\}_{i,j,k}, P_0) \leftarrow \text{PIOP.Prover.Format}(P_{\text{wit}}, P_{\text{rnd}})$
 - 2: $(\gamma', \gamma) \leftarrow \text{ExpandBatchingChallenge}(h_{\text{lines}})$ $\triangleright \gamma \in \mathbb{F}_{2^\lambda}^{\lceil \frac{n-k}{\lambda} \rceil}, \gamma' \in \mathbb{F}_{2^\lambda}^w$
 - 3: $(\{h_j^{[\gamma]}\}_j, y^{[\gamma]}) \leftarrow \text{BatchLinearEquations}(\gamma, \mathbf{H}', \mathbf{y})$
 - $\triangleright$ Compute the degree- α output polynomial $P_\alpha(X)$.
 - 4: $P_\alpha(X) \leftarrow P_0(X) \cdot X - y^{[\gamma]}$
 - 5: **for** i from 0 to $w-1$ **do**
 - 6: $P_\alpha(X) \leftarrow P_\alpha(X) + \text{PIOP.Prover.MuxTree}\left((h_{i \cdot m}^{[\gamma]}, \dots, h_{(i+1) \cdot m-1}^{[\gamma]}), \{P_{w_{i,j,k}}(X)\}_{j,k}\right)$
 - 7: $P_\alpha(X) \leftarrow P_\alpha(X) + \gamma'_i \cdot \sum_{j=0}^{d-1} \xi^{32(j-1)} \cdot \text{PIOP.Prover.UnitaryCheck}(\mu_j, \{P_{w_{i,j,k}}(X)\}_k)$
 - 8: **return** $P_\alpha(X)$
-

Algorithm 18 describes the verifier's computation in the PIOP protocol, namely how the verifier computes the evaluation $p_\alpha = P_\alpha(\Delta)$ from the evaluations $P_0(\Delta)$ and $\mathbf{P}_{\text{wit}}(\Delta)$. The first step consists in formatting the input evaluations using the routine **PIOP.Verifier.Format**: it parses the evaluation vector $\mathbf{p}_{\text{wit}} = \mathbf{P}_{\text{wit}}(\Delta)$ as $\{p_{w_{i,j,k}}\}_{i,j,k} = \{P_{w_{i,j,k}}(\Delta)\}_{i,j,k}$ and builds the evaluation $p_0 = P_0(\Delta)$ from the evaluation vector $\mathbf{p}_{\text{rnd}} = P_{\text{rnd}}(\Delta)$. Next, the verifier challenge (γ', γ) , i.e., the randomness for batching the relations, is generated by hashing the BLC commitment h_{lines} . Then the batched field-embedded elements $\{h_j^{[\gamma]}\}$ and $y^{[\gamma]}$ are computed (see Equation 9). Finally, the evaluation $p_\alpha = P_\alpha(\Delta)$ is computed in the same way as the polynomial P_α on the prover side (see above equation).

Algorithm 18 `PIOP.Verifier.Run`

Input: a field element $\Delta \in \mathbb{F}_{2^\lambda}$, two vectors $p_{\text{wit}} \in \mathbb{F}_{2^\lambda}^{|\text{wit}|}$ and $p_{\text{rnd}} \in \mathbb{F}_{2^\lambda}^{|\text{rnd}|}$, a hash digest $h_{\text{lines}} \in \{0, 1\}^{2^\lambda}$ and the regular syndrome instance $(\mathbf{H}', \mathbf{y})$.

Output: a field element $p_\alpha \in \mathbb{F}_{2^\lambda}$

- 1: $(\{p_{w_{i,j,k}}\}_{i,j,k}, p_0) \leftarrow \text{PIOP.Verifier.Format}(p_{\text{wit}}, p_{\text{rnd}})$
- 2: $(\gamma', \gamma) \leftarrow \text{ExpandBatchingChallenge}(h_{\text{lines}})$ $\triangleright \gamma \in \mathbb{F}_{2^\lambda}^{\lceil \frac{n-k}{\lambda} \rceil}, \gamma' \in \mathbb{F}_{2^\lambda}^w$
- 3: $(\{h_j^{[\gamma]}\}_j, y^{[\gamma]}) \leftarrow \text{BatchLinearEquations}(\gamma, \mathbf{H}', \mathbf{y})$
- $\triangleright$ Compute the evaluation $p_\alpha := P_\alpha(\Delta)$.
- 4: $p_\alpha \leftarrow p_0 \cdot \Delta - y^{[\gamma]}$
- 5: **for** i from 0 to $w - 1$ **do**
- 6: $p_\alpha \leftarrow p_\alpha + \text{PIOP.Verifier.MuxTree}((h_{i \cdot m}^{[\gamma]}, \dots, h_{(i+1) \cdot m - 1}^{[\gamma]}), \{p_{w_{i,j,k}}\}_{j,k})$
- 7: $p_\alpha \leftarrow p_\alpha + \gamma'_i \cdot \sum_{j=1}^d \xi^{32(j-1)} \cdot \text{PIOP.Verifier.UnitaryCheck}(\mu_j, \{p_{w_{i,j,k}}\}_k)$
- 8: **return** p_α

Algorithm 19 `BatchLinearEquations`

Input: $\gamma, \mathbf{H}', \mathbf{y}$

- 1: **for** j from 0 to $n - 1$ **do**
- 2: $h_j^{[\gamma]} \leftarrow \gamma^\top \cdot \Phi(\mathbf{h}_j)$ $\triangleright h_j^{[\gamma]} \in \mathbb{F}_{2^\lambda}, \mathbf{h}_j$ is the j^{th} column of $\mathbf{H} = [\mathbf{H}' \mid I_{n-k}]$
- 3: $y^{[\gamma]} \leftarrow \gamma^\top \cdot \Phi(\mathbf{y})$ $\triangleright y^{[\gamma]} \in \mathbb{F}_{2^\lambda}$
- 4: **return** $\{h_j^{[\gamma]}\}_j, y^{[\gamma]}$

4 Parameters and performances

In this section, we propose several parameter sets for the SD-in-the-Head signature scheme. As explained hereafter, those parameters have been selected to meet the security categories I, III and V defined by the NIST while targeting good performances (signature size and running times).

4.1 Selection of parameters

RSD parameters. The RSD parameters include three values: n the length of the code, k_{RSD} the dimension of the code and w the weight of the secret vector. We recall that in an RSD instance, the secret vector is made of w blocks of size $\frac{n}{w}$ and weight 1.

For $\theta \in [1, \infty)$, we define $k_{\text{RSD}}^{\max}$, the largest RSD dimension leading to RSD density $\leq 1/\theta$:

$$k_{\text{RSD}}^{\max} := n - \left\lfloor w \cdot \log_2 \left(\frac{n}{w} \right) - \log_2 \theta \right\rfloor .$$

We further define k_{SD} as the largest SD dimension leading to SD density ≤ 1 (for non-regular SD with parameters n and w):

$$k_{\text{SD}} := n - \left\lfloor \log_2 \binom{n}{w} \right\rfloor .$$

According to the security reduction provided in [Section 5.1](#), for any k_{RSD} such that

$$k_{\text{SD}} \leq k_{\text{RSD}} \leq k_{\text{RSD}}^{\max}$$

an algorithm solving an (n, k_{RSD}, w) -RSD instance in complexity λ' bits implies an algorithm solving an (n, k_{SD}, w) -SD instance with complexity

$$\lambda' + \log_2 \left(\frac{\binom{n}{w}}{\left(\frac{n}{w}\right)^w} \right) + \frac{1}{\theta \ln 2} . \quad (10)$$

Parameters are chosen such that for $\lambda' \in \{143, 207, 272\}$ (corresponding to NIST Categories I, III and V), the best binary SD attacks for an (n, k_{SD}, w) -instance have a complexity greater than (10). Thanks to our reduction ([Theorem 5.1](#)), finding an attack with complexity less than λ' for an (n, k_{RSD}, w) -RSD instance would mean improving over the best known attacks for (n, k_{SD}, w) -SD instances, the currently hardest type of SD instances (as being on the Gilbert-Varshamov bound). In practice for implementation reasons k_{RSD} is chosen as the greatest value such that $n - k_{\text{RSD}}$ is a multiple of 8 and $k_{\text{SD}} \leq k_{\text{RSD}} \leq k_{\text{RSD}}^{\max}$.

Concretely, for our parameter selection, we proceeded as follows. For each w , we find the smallest n that is a multiple of w so that the (n, k_{SD}, w) -SD instance achieve security (10). We then set k_{RSD} as the largest number $\leq k_{\text{RSD}}^{\max}$ such that the codimension $n - k_{\text{RSD}}$ is an exact multiple of 8. Then for each degree d (we test them all), we consider the mux arities $\mu = (\mu_1, \dots, \mu_d)$ that minimizes the witness size w (the greedy algorithm that selects $\mu_1 = \lceil n/w \rceil^{1/d}$ and continues recursively for $(\mu_2, \dots, \mu_d)$ on $\lceil n/w/\mu_1 \rceil$ yields the optimum). We finally set the degree d as the one that minimizes $w + \lambda \cdot (d - 1)$, and thus, the total signature size. The results are provided in [Table 3](#) hereafter. The best parameters are obtained for arities $\mu = (4, 4, 4, 4)$ or $\mu = (4, 4, 4, 3)$, inducing block sizes of either 256 or 192, with rather large values of n .

Proof system parameters. For each security level, we consider two variants: a “short” variant with larger GGM trees (decreasing the number of repetitions τ and hence the signature size), and a “fast” variant with smaller GGM trees (making the computation faster). For the “short” variant, we use $N = 2^{11}$ for Category I and $N = 2^{12}$ for Categories III and V. For the “fast” variant, we use $N = 2^8$ for all three categories. To achieve a soundness of λ bits, with $\lambda \in \{128, 192, 256\}$ (for Categories I, III and V), we must select the number of repetitions τ , and the grinding parameter w_{pow} , to satisfy:

$$\tau \cdot \kappa - \log_2(d) + w_{\text{pow}} \geq \lambda ,$$

where $\kappa = \log_2(N)$. Concretely, we choose τ such that $w_{\text{pow}} := \lambda + \log_2(d) - \tau \cdot \kappa$ is a small integer, typically lower than 5 for the “fast” variants, and up to 10 for the “short” variants. This strategy yields $w_{\text{pow}} = 2$ for the “fast” variants at each security level while we get $w_{\text{pow}} \in \{2, 6, 9\}$ for the “short” variant. Finally, we fix the value of T_{open} based on experiments to obtain good trade-offs between sizes and performances.

The proof system relies on the field on a λ -bit field, namely $\mathbb{F}_{2^{128}}$, $\mathbb{F}_{2^{192}}$ and $\mathbb{F}_{2^{256}}$ depending on the security level. Table 2 summarizes the field extensions that we use in our instances.

Table 2: Definition of field extensions.

Field ($\mathbb{F}_{2^\lambda}$)	Field extension
$\mathbb{F}_{2^{128}}$	$\mathbb{F}_2[\xi]/\langle \xi^{128} + \xi^7 + \xi^2 + \xi^1 + 1 \rangle$
$\mathbb{F}_{2^{192}}$	$\mathbb{F}_2[\xi]/\langle \xi^{192} + \xi^7 + \xi^2 + \xi^1 + 1 \rangle$
$\mathbb{F}_{2^{256}}$	$\mathbb{F}_2[\xi]/\langle \xi^{256} + \xi^{10} + \xi^5 + \xi^2 + 1 \rangle$

4.2 Keys and signature sizes

Public key. The public key has format $pk := (\text{seed}_{\text{pk}}, \mathbf{y})$; consisting of a λ -bit seed seed_{pk} used to generate the matrix $\mathbf{H}'$, and a serialized vector $\mathbf{y} := \mathbf{H}\mathbf{x} \in \mathbb{F}_2^{n-k}$ corresponding to the syndrome. The public key has a total size (in bytes) of

$$|pk| = \frac{1}{8}(\lambda + n - k) .$$

Secret key. The secret key has format $sk := (\text{seed}_{\text{pk}}, \mathbf{y}, \text{wit}, \text{seed}_{\text{sk}})$; consisting of the same seed_{pk} and $\mathbf{y}$ as the public key, as well as the λ -bit seed seed_{sk} and the serialized witness wit . The latter is made of w sets of d truncated elementary vectors of size $\mu_1 - 1, \dots, \mu_d - 1$. Thus, the size of the secret key (in bytes) is

$$|sk| = \left\lceil \frac{1}{8} \left(2\lambda + (n - k) + w \cdot \sum_{i=1}^d (\mu_i - 1) \right) \right\rceil .$$

Signature size. The size (in bits) of a signature is:

$$\begin{aligned}
 |\sigma| &= 3\lambda && \rightarrow \text{salt}, h_{\text{piop}} \\
 &+ (\tau - 1) \cdot (|\text{wit}|_2 + (d - 1)\lambda + (\lambda + B)) && \rightarrow \text{aux} \\
 &+ (\lambda + B) && \rightarrow \alpha'_{\text{plain}} \\
 &+ |\text{wit}|_2 && \rightarrow \Delta \text{wit} \\
 &+ d \cdot \lambda && \rightarrow (\alpha_1, \dots, \alpha_d) \\
 &+ \lambda \cdot T_{\text{open}} + \tau \cdot (2\lambda) + 32 && \rightarrow \text{pdecom}
 \end{aligned}$$

with $|\text{wit}|_2 := \text{ceil}_8(w \cdot \sum_{i=1}^d (\mu_i - 1))$, where $\text{ceil}_8(x) = 8 \cdot \lceil x/8 \rceil$.

4.3 Selected parameters

The signature parameters of our proposed instances are summarized in Table 3 and in Table 4 for the different security categories. Table 3 gives the syndrome decoding parameters which are common to both trade-offs while Table 4 gives the proof system parameters and associated sizes.

Table 3: RSD parameters of SD-in-the-Head-2.

Parameter Sets	NIST Security		RSD Parameters					Modeling
	Category	Bits	q	n	$(n - k_{\text{RSD}})$	k_{RSD}	w	μ
SDitH2-L1-gf2	I	143	2	10 360	432	9 928	56	[4,4,4,3]
SDitH2-L3-gf2	III	207	2	18 396	592	17 804	73	[4,4,4,4]
SDitH2-L5-gf2	V	272	2	19 864	800	19 064	104	[4,4,4,3]

Table 4: Proof system parameters of SD-in-the-Head-2, with key and signature sizes.

Parameter Set	Proof System Parameters					Sizes (Bytes)				
	τ	κ	w_{pow}	T_{open}	B	pk	sk	Sig. Avg	Sig. Max	
SDitH2-L1-gf2-short	11	11	9	107	16	70	163	3 705	3 705	
SDitH2-L1-gf2-fast	16	8	2	101	16	70	163	4 484	4 484	
SDitH2-L3-gf2-short	16	12	2	157	16	98	232	7 964	7 964	
SDitH2-L3-gf2-fast	24	8	2	153	16	98	232	9 916	9 916	
SDitH2-L5-gf2-short	21	12	6	216	16	132	307	14 121	14 121	
SDitH2-L5-gf2-fast	32	8	2	207	16	132	307	17 540	17 540	

4.4 Benchmarks

Table 5 and Table 6 provide benchmarks for the key generation, signature and verification algorithms of SD-in-the-Head-2 on a laptop and a cloud server respectively. The provided timings are median over 200 runs. For Category I, the signing and verification algorithms run in 2–3 ms with the “fast” variant and in 6–9 ms with the “short” one.

Table 5: Timings on a laptop with 12th Gen Intel Core i7-1260P (median after 200 runs).

	KeyGen	Sign	Verif
SDitH2-L1-gf2-short	0.63 ms	6.73 ms	6.04 ms
SDitH2-L1-gf2-fast	0.74 ms	2.01 ms	1.79 ms
SDitH2-L3-gf2-short	3.02 ms	42.26 ms	39.83 ms
SDitH2-L3-gf2-fast	1.56 ms	6.36 ms	5.75 ms
SDitH2-L5-gf2-short	1.55 ms	60.48 ms	57.23 ms
SDitH2-L5-gf2-fast	1.82 ms	9.42 ms	8.70 ms

Table 6: Timings on a cloud server with AMD EPYC 7B13 @ 2.45GHZ (median after 200 runs).

	KeyGen	Sign	Verif
SDitH2-L1-gf2-short	0.61ms	9.33ms	8.18ms
SDitH2-L1-gf2-fast	0.59ms	2.96ms	2.67ms
SDitH2-L3-gf2-short	1.71ms	44.54ms	41.38ms
SDitH2-L3-gf2-fast	1.61ms	7.86ms	7.11ms
SDitH2-L5-gf2-short	1.94ms	62.18ms	57.56ms
SDitH2-L5-gf2-fast	1.92ms	11.17ms	10.23ms

5 Security

5.1 SD to RSD security reduction

We provide hereafter a security reduction from SD to RSD. Namely, any SD instance can be solved using an RSD solver. The latter must be called a certain number of times which implies a security gap between the two instances. This gap must be compensated by increasing the RSD parameters. As explained in [Section 4.1](#), this is precisely the approach we followed to select the RSD parameters of SD-in-the-Head-2.

Theorem 5.1. *Let $\mathbb{F}$ be a finite field. Let $\theta \in [1, \infty)$. Let $n, k_{\text{SD}}, k_{\text{RSD}}, w$ be positive integers such that $k_{\text{SD}} \leq k_{\text{RSD}} \leq n$, $w < n$, $w \mid n$, and*

$$k_{\text{RSD}} \leq n - w \cdot \log_2 \left(\frac{n}{w} \right) - \log_2 \theta .$$

Let $\mathcal{A}_{\text{RSD}}$ be an algorithm solving a random $(\mathbb{F}, n, k_{\text{RSD}}, w)$ -instance of the regular syndrome decoding problem in time t with success probability ε_{RSD} . Then there exists an algorithm $\mathcal{A}_{\text{SD}}$ solving a random $(\mathbb{F}, n, k_{\text{SD}}, w)$ -instance of the standard syndrome decoding problem in time t with probability ε_{SD} , where

$$\varepsilon_{\text{SD}} \geq e^{-1/\theta} \cdot \frac{\left(\frac{n}{w}\right)^w}{\binom{n}{w}} \cdot \varepsilon_{\text{RSD}} .$$

Remark 1. *In the above theorem, for $\theta = 1$, the constraint $k_{\text{RSD}} \leq n - w \cdot \log_2 \left(\frac{n}{w} \right)$ implies that the regular SD instance $(\mathbb{F}, n, k_{\text{RSD}}, w)$ has density at most 1. Taking a greater θ implies a lower density.*

Proof. We adapt the proof of [\[FJR22\]](#) to our context. To prove the theorem, we build an algorithm $\mathcal{A}_{\text{SD}}$ to solve the traditional SD problem of parameters (n, k, w) using an algorithm $\mathcal{A}_{\text{RSD}}$ which solves the regular SD problem with the same parameters.

Algorithm $\mathcal{A}_{\text{SD}}$ (on input an SD instance $(\mathbf{H}, \mathbf{y})$):

1. Sample a permutation σ of $\{1, \dots, n\}$.
2. Permute the columns of $\mathbf{H}$ using σ to get $\hat{\mathbf{H}}$.
3. Remove the $k_{\text{RSD}} - k_{\text{SD}}$ last rows of $\hat{\mathbf{H}}$ and the last $k_{\text{RSD}} - k_{\text{SD}}$ coordinates of $\mathbf{y}$ to obtain $\hat{\mathbf{H}}_{\text{tr}}$ and $\mathbf{y}_{\text{tr}}$.
3. Run $\mathcal{A}_{\text{RSD}}$ on input $(\hat{\mathbf{H}}_{\text{tr}}, \mathbf{y}_{\text{tr}})$ to get $\hat{\mathbf{x}}$.
4. If $\hat{\mathbf{x}} = \perp$, return $\perp$.
5. If $\hat{\mathbf{H}}\hat{\mathbf{x}} \neq \mathbf{y}$, return $\perp$.
6. Permute the coordinates of $\hat{\mathbf{x}}$ using σ^{-1} to get $\mathbf{x}$.
7. Return $\mathbf{x}$.

The probability to transform an SD instance into a regular SD instance in Step 2 is $\left(\frac{n}{d}\right)^w / \binom{n}{w}$.

Thus we have

$$\begin{aligned}
\varepsilon_{\text{SD}} &:= \Pr[\mathcal{A}_{\text{SD}}(\mathbf{H}, \mathbf{y}) \neq \perp] \\
&\geq \Pr[\mathcal{A}_{\text{SD}}(\mathbf{H}, \mathbf{y}) \neq \perp \cap (\hat{\mathbf{H}}, \mathbf{y}) \text{ is an RSD instance}] \\
&= \frac{\binom{n}{w}^w}{\binom{n}{w}} \cdot \Pr[\mathcal{A}_{\text{SD}}(\mathbf{H}, \mathbf{y}) \neq \perp \mid (\hat{\mathbf{H}}, \mathbf{y}) \text{ is an RSD instance}] \\
&= \frac{\binom{n}{w}^w}{\binom{n}{w}} \cdot \Pr[\mathcal{A}_{\text{RSD}}(\hat{\mathbf{H}}_{\text{tr}}, \mathbf{y}_{\text{tr}}) \neq \perp \text{ and } \hat{\mathbf{H}}\hat{\mathbf{x}} = \mathbf{y} \mid (\hat{\mathbf{H}}, \mathbf{y}) \text{ is an RSD instance}] \\
&= \frac{\binom{n}{w}^w}{\binom{n}{w}} \cdot \Pr[\mathcal{A}_{\text{RSD}}(\hat{\mathbf{H}}_{\text{tr}}, \mathbf{y}_{\text{tr}}) \neq \perp \mid (\hat{\mathbf{H}}, \mathbf{y}) \text{ is an RSD instance}] \\
&\quad \cdot \Pr[\hat{\mathbf{H}}\hat{\mathbf{x}} = \mathbf{y} \mid (\hat{\mathbf{H}}, \mathbf{y}) \text{ is an RSD instance, } \mathcal{A}_{\text{RSD}}(\hat{\mathbf{H}}_{\text{tr}}, \mathbf{y}_{\text{tr}}) \neq \perp]
\end{aligned}$$

By definition we have:

$$\Pr[\mathcal{A}_{\text{RSD}}(\hat{\mathbf{H}}_{\text{tr}}, \mathbf{y}_{\text{tr}}) \neq \perp \mid (\hat{\mathbf{H}}, \mathbf{y}) \text{ is an RSD instance}] = \varepsilon_{\text{RSD}} .$$

On the other hand,

$$\begin{aligned}
&\Pr[\hat{\mathbf{H}}\hat{\mathbf{x}} = \mathbf{y} \mid (\hat{\mathbf{H}}, \mathbf{y}) \text{ is an RSD instance, } \mathcal{A}_{\text{RSD}}(\hat{\mathbf{H}}_{\text{tr}}, \mathbf{y}_{\text{tr}}) \neq \perp] \\
&\geq \Pr[(\hat{\mathbf{H}}_{\text{tr}}, \mathbf{y}_{\text{tr}}) \text{ has a single RSD solution} \mid \mathcal{A}_{\text{RSD}}(\hat{\mathbf{H}}_{\text{tr}}, \mathbf{y}_{\text{tr}}) \neq \perp] .
\end{aligned}$$

Indeed, if $(\hat{\mathbf{H}}_{\text{tr}}, \mathbf{y}_{\text{tr}})$ has a single RSD solution, then given that $(\hat{\mathbf{H}}, \mathbf{y})$ is an RSD instance, we have that the right solution is returned by $\mathcal{A}_{\text{RSD}}$. Now, we heuristically have:

$$\Pr[(\hat{\mathbf{H}}_{\text{tr}}, \mathbf{y}_{\text{tr}}) \text{ has a single RSD solution} \mid \mathcal{A}_{\text{RSD}}(\hat{\mathbf{H}}_{\text{tr}}, \mathbf{y}_{\text{tr}}) \neq \perp] \approx \left(1 - \frac{1}{nb_y}\right)^{nb_x} ,$$

which is the probability that no other $\hat{\mathbf{x}}$ lead to the same $\mathbf{y}_{\text{tr}}$, with $nb_x = \binom{n}{w}^w - 1$ and $nb_y = 2^{n-k_{\text{RSD}}} \leq \theta \cdot \binom{n}{w}^w$, implying

$$\left(1 - \frac{1}{nb_y}\right)^{nb_x} \approx e^{-1/\theta} .$$

□

Informally, the above result holds because an instance of the standard SD problem is an instance of the regular syndrome decoding problem with probability $\binom{n}{w}^w / \binom{n}{w}$. Moreover, a standard syndrome decoding instance can be “randomized” and input to the regular adversary as much as desired.

All the regular SD instances used in SD-in-the-Head are chosen such that the corresponding standard SD instance achieves a security level which compensates the degradation. We stress that this might be overly conservative.

5.2 Attacks against the SD problem

The binary SD problem has been studied for many years: the first attack was proposed by Prange in the 60’s. Later further attacks were proposed by Stern and Dumer at the turning of the 80’s and the 90’s. Many new attacks came out since 2010: Becker, Joux, May and

Meurer [BJM⁺12] in 2012, then May and Ozerov [MO15] in 2015 and Both and May [BM18] in 2018. Eventually a cryptographic estimator was proposed by Esser and Bellini in 2022 [EB22].

While original attacks did not use much memory, recent attacks make extensive use of memory. In this context, the cost of memory accesses (as a function of the memory size) is an essential parameter [EB22]. This cost can be considered to be constant, logarithmic, cubic root or square root. While a constant-time memory-access cost might seem very optimistic from the attacker standpoint (and hence over conservative), a square root access might seem too pessimistic (hence risky in terms of security). Unfortunately, there is no wide consensus on which cost is the most practically relevant.² It is interesting to notice that considering a cubic or square root memory-access cost (which is theoretically meaningful) significantly limit the impact of recent attacks whose efficiency relies on the availability of a large memory.

For our SD security estimates, we made the conservative choices of considering a logarithmic memory-access cost and a memory size limited to 2^{143} for Category I, and 2^{160} for Categories III and V (which roughly corresponds to the number of atoms on earth). To derive our concrete parameters, we used the “Syndrome Decoding Estimator”, an open-source tool available at: https://github.com/Crypto-TII/syndrome_decoding_estimator.

5.3 Unforgeability

The SD-in-the-Head signature scheme aims at providing *unforgeability against chosen message attacks* (EUF-CMA). In this setting, the adversary is given a public key pk and they can ask an oracle (called the *signature oracle*) to sign messages $(msg_1, \dots, msg_r)$ that they can select at will. The goal of the adversary is to generate a pair (msg, σ) such that msg is not one of requests to the signature oracle and such that σ is a valid signature of msg with respect to pk .

Our security statement is based on the following assumptions:

- **SD hardness.** Solving the considered SD instance is (ϵ_{SD}, t) -hard for some (ϵ_{SD}, t) which are implicit functions of the security parameter λ . Formally, any adversary $\mathcal{A}$ on input a random SD instance and running in time at most t has probability at most ϵ_{SD} to output the solution of the input instance.
- **Random Oracle Modem (ROM).** Our security statement holds in the ROM where the (extendable-output) hash function Hash is modelled as a random oracle.
- **Ideal Cipher Model (ICM).** Our security statement holds in the ICM where the block cipher Enc is modelled as an ideal cipher.

Based on the ROM and the ICM, the EUF-CMA security of SD-in-the-Head holds from the soundness and zero-knowledge properties of the underlying ZK-PoK (which are overviewed in Section 2). The formal EUF-CMA security proof of SD-in-the-Head will be added to a future version of the specification. It will heavily rely on usual techniques for MPC-in-the-Head signature schemes with GGM trees.

²See, e.g., https://groups.google.com/a/list.nist.gov/g/pqc-forum/c/EiwxGnfQgec/m/xBky_FKFDgAJ

6 Variants

In this section, we outline two possible variants of SD-in-the-Head-2, namely a variant using the base field $\mathbb{F}_{256}$ and a variant using the Threshold-Computation-in-the-Head framework. We might consider including these variants as formal instances of the scheme in the future.

6.1 The $\mathbb{F}_{256}$ variant

In the previous sections, we focused on the RSD problem over the binary field $\mathbb{F}_2$. We might consider larger fields $\mathbb{F}_q$, with $q > 2$. Given a matrix $\mathbf{H} \in \mathbb{F}_q^{(n-k) \times n}$ and a syndrome vector $\mathbf{y} \in \mathbb{F}_q^{n-k}$, the RSD problem over $\mathbb{F}_q$ consists in finding a vector $\mathbf{x} \in \mathbb{F}_q^n$ such that

- $\mathbf{x}$ satisfies the linear relation $\mathbf{y} = \mathbf{H}\mathbf{x}$, and
- $\mathbf{x}$ is regular, meaning that it is the concatenation of w scaled elementary vectors $e_1, \dots, e_w$ of size $\frac{n}{w}$ (*i.e.* vector with $\frac{n}{w} - 1$ coefficients 0 and one non-zero random coefficient).

The main difference when working on a larger field is that the non-zero coordinates of the solution $\mathbf{x}$ are not necessary 1, they can be other non-zero values. In this context, the witness wit should contain the value of those coordinates, together with their positions. To be able to embed the tensor-product decomposition of the RSD solution over $\mathbb{F}_2$ while working over $\mathbb{F}_q$, we need to consider fields of characteristic 2, *i.e.* binary field extensions. The size of the secret key (in bytes) is then

$$|\text{sk}| = \left\lceil \frac{1}{8} (2\lambda + (n - k) \cdot \log_2(q) + w \cdot \sum_{i=1}^d (\mu_i - 1) + \underbrace{w \cdot \log_2(q)}_{\text{Values of the non-zero coordinates}}) \right\rceil,$$

while the public key has a total size (in bytes) of

$$|\text{pk}| = \left\lceil \frac{1}{8} (\lambda + (n - k) \cdot \log_2(q)) \right\rceil.$$

The polynomial constraints that the witness should satisfy are very similar to those of the $\mathbb{F}_2$ case. The only difference comes from the fact that we need to scale each chunk e_i by the non-zero value. In practice, this can be done just after applying the mux tree at each chunk. The size (in bits) of a signature is:

$$\begin{aligned} |\sigma| &= 3\lambda && \rightarrow \text{salt}, h_{\text{piop}} \\ &+ (\tau - 1) \cdot (|\text{wit}|_2 + (d - 1)\lambda + (\lambda + B)) && \rightarrow \text{aux} \\ &+ (\lambda + B) && \rightarrow \alpha'_{\text{plain}} \\ &+ |\text{wit}|_2 && \rightarrow \Delta \text{wit} \\ &+ d \cdot \lambda && \rightarrow (\alpha_1, \dots, \alpha_d) \\ &+ \lambda \cdot T_{\text{open}} + \tau \cdot (2\lambda) + 32 && \rightarrow \text{pdecom} \end{aligned}$$

with $|\text{wit}|_2 := \text{ceil}_8(w \cdot \log_2(q) + w \cdot \sum_{i=1}^d (\mu_i - 1))$, where $\text{ceil}_8(x) = 8 \cdot \lceil x/8 \rceil$.

We exhibit some instances over $\mathbb{F}_{256}$ in Table 3 and in Table 4 for the different security categories. Table 3 gives the syndrome decoding parameters which were obtained following the approach described in Section 4.1 adapted to $\mathbb{F}_{256}$. Table 4 gives the proof system parameters

(which are the same as for the $\mathbb{F}_2$ instances) and the associated sizes. We observe that we get similar sizes than over $\mathbb{F}_2$.

While working on $\mathbb{F}_2$ can be considered as a more conservative choice than working on $\mathbb{F}_{256}$, the latter has the main advantage to lead to a lighter scheme. In particular, the PIOP computation is expected to be significantly faster over $\mathbb{F}_{256}$ as requiring much fewer large-field multiplications. This is because the manipulated vectors are shorter while, on the other hand, the large field remains the same (i.e. $\mathbb{F}_{2^\lambda}$). Moreover, the uncompressed matrix $\mathbf{H}'$ is smaller over $\mathbb{F}_{256}$. For instance, for Category I, we have 560 KB for $\mathbf{H}'$ on $\mathbb{F}_2$ versus 140 KB on $\mathbb{F}_{256}$.

Table 7: RSD parameters of SD-in-the-Head-2 for $\mathbb{F}_{256}$.

Parameter Sets	NIST Security		RSD Parameters					Modeling
	Category	Bits	q	n	$(n - k_{\text{RSD}})$	k_{RSD}	w	μ
SDitH2-L1-gf256	I	143	256	2 176	64	2 112	34	[4,4,4]
SDitH2-L3-gf256	III	207	256	3 200	96	3 104	50	[4,4,4]
SDitH2-L5-gf256	V	272	256	4 224	120	4 104	66	[4,4,4]

Table 8: Proof system parameters of SD-in-the-Head-2 over $\mathbb{F}_{256}$, with key and signature sizes.

Parameter Set	Proof System Parameters					Sizes (Bytes)			
	τ	κ	w	T_{open}	B	pk	sk	Sig.	Baseline ($\mathbb{F}_2$)
SDitH2-L1-gf256-short	11	11	9	107	16	80	169	3 661	3 705
SDitH2-L1-gf256-fast	16	8	2	101	16	80	169	4 420	4 484
SDitH2-L3-gf256-short	16	12	2	157	16	120	251	7 916	7 964
SDitH2-L3-gf256-fast	24	8	2	153	16	120	251	9 844	9 916
SDitH2-L5-gf256-short	21	12	6	216	16	152	325	14 079	14 121
SDitH2-L5-gf256-fast	32	8	2	207	16	152	325	17 476	17 540

6.2 The TCitH variant

As explained in Section 2.1, an alternative choice to the VOLE-in-the-Head framework [BBD⁺23] is the Threshold-Computation-in-the-Head (TCitH) framework [FR23a]. The TCitH-based commitment scheme enables the prover/signer to open one evaluation among only N , while the computational complexity of the commitment procedure is linear in N . While the PIOP soundness error is d/N^τ for VOLEitH, it is $(d/N)^\tau$ for TCitH with τ parallel repetitions, which means that we must ensure $(d/N)^\tau \cdot 2^{-w_{\text{pow}}} \leq 2^{-\lambda}$ to achieve a λ -bit security. The size (in bits) of a TCitH-based SD-in-the-Head-2 signature is:

$$\begin{aligned}
|\sigma| &= 3\lambda && \rightarrow \text{salt}, h_{\text{piop}} \\
&+ \tau \cdot |\text{wit}|_2 && \rightarrow \Delta \text{wit} \\
&+ \tau \cdot (d - 1) \cdot \text{ceil}_\kappa(\lambda) && \rightarrow (\alpha_2, \dots, \alpha_d) \\
&+ \lambda \cdot T_{\text{open}} + \tau \cdot (2\lambda) + 32 && \rightarrow \text{pdecom}
\end{aligned}$$

with $|\text{wit}|_2 := \text{ceil}_8(w \cdot \sum_{i=1}^d (\mu_i - 1))$, where $\text{ceil}_n(x) = n \cdot \lceil x/n \rceil$.

Table 9 gives the TCitH-based proof system parameters and the associated signature sizes. Let us stress that the key generation is the same in both approaches and so the key sizes are those in Table 4. We can observe that the signature sizes are larger when using the TCitH framework. This comes from the required increase of τ to maintain an equivalent security. This is particularly true for SD-in-the-Head-2, for which the RSD modeling leads to degree-4 (*i.e.* $d = 4$) constraints, while most of the schemes in the literature consider only degree-2. This size increasing is partly compensated by the fact that TCitH does not include the communication cost induced by the VOLEitH consistency check.

While it leads to larger signatures, the main advantage of the TCitH variant is to be structurally simpler: it does not have consistency check, it is derived from a 5-round interactive protocol (while the VOLEitH framework gives 7-round protocol), and it does not require to work in a large field extension.

Table 9: Proof system parameters of the TCitH-based variant of SD-in-the-Head-2, with signature sizes. Key sizes are similar to our main VOLEitH-based instances.

Parameter Set	Proof System Parameters				Sizes (Bytes)	
	τ	κ	w_{pow}	T_{open}	Sig.	Baseline (VOLEitH)
SDitH2-TCitH-L1-gf2-short	14	11	2	125	4 271	3 705 (-13%)
SDitH2-TCitH-L1-gf2-fast	21	8	2	135	5 509	4 484 (-19%)
SDitH2-TCitH-L3-gf2-short	19	12	2	185	8 426	7 964 (-5%)
SDitH2-TCitH-L3-gf2-fast	31	8	6	212	11 374	9 916 (-13%)
SDitH2-TCitH-L5-gf2-short	25	12	6	265	15 618	14 121 (-10%)
SDitH2-TCitH-L5-gf2-fast	42	8	4	280	19 968	17 540 (-12%)

7 Advantages and limitations

In this section we describe some advantages and limitations of the SD-in-the-Head signature scheme. The bottom line is that it provides both conservative security *and* relatively small signatures compared to current PQC standards.

7.1 Advantages of SD-in-the-Head

Conservative hardness assumption. Our signature scheme is based on the presumably hardest problem in code-based cryptography: the *unstructured* binary Syndrome Decoding (SD) problem for random linear codes. This problem is known to be NP-hard and the cryptanalysis state-of-the-art has been stable and well-established for decades.

Adaptive and tunable parameters. MPCitH enables us to tailor parameters, in particular the size of GGM trees (i.e., the size of the small evaluation domain), meaning that we can provide a variety of parameter sets tailored to different use cases similarly to SPHINCS⁺. This is illustrated by our ‘fast’ and ‘short’ parameter sets that provide two different signature sizes/performance trade-offs. In addition, the size of the signature is composed of two parts: a part related to GGM trees and a part related to the SD instance. The latter part is not the bottleneck so that increasing the size of the SD parameters (and hence the associated SD security) only has a moderate impact on the global signature size.

Small code-based signatures. The SD-in-the-Head signature scheme achieves among the smallest code-based signatures to-date, which does not come at the cost of a large public key.

Small key sizes. Both the secret key and public key sizes are much smaller in comparison to the lattice-based signature standards, and compete with SHL-DSA. In particular, the public key, which is often transported with the signature (e.g., certificates in TLS), is between 120-240 bytes across all security levels for both variants.

Size of public key and signature. SD-in-the-Head offers competitive signature sizes along with very small public keys, which yields a competitive signature + public key size. For NIST security level I, the sum of the signature and public key sizes of SD-in-the-Head gives 3.8 kB, which is comparable or smaller than the post-quantum NIST standards ML-DSA (Dillitium) and SLH-DSA (SPHINCS⁺) with 3.7 kB and 7.8 kB respectively.

7.2 Limitations of SD-in-the-Head

Quadratic growth w.r.t. the security level. As other MPCitH schemes, or, more generally, as other schemes applying the Fiat-Shamir transform to a parallelly repeated ZK-PoK with non-negligible soundness error, SD-in-the-Head suffers a quadratic growth of its signature size.

Efficiency. MPCitH-like schemes require the generation of lots of pseudorandom objects, which makes them slow in comparison to other schemes such as the NIST post-quantum standard ML-DSA. Nonetheless, the efficiency of SD-in-the-Head is competitive when compared with many other post-quantum signature schemes.

Low-cost devices and embedded systems. SD-in-the-Head might be particularly heavy for low-cost devices such as smart cards or embedded systems, although it has the potential to perform well on hardware as being highly parallelizable.

References

- [AGH⁺22] C. Aguilar-Melchor, N. Gama, J. Howe, A. Hülsing, D. Joseph, and D. Yue. The return of the SDitH. Cryptology ePrint Archive, Report 2022/1645, 2022 (cited on page 2).
- [AGH⁺23] C. Aguilar-Melchor, N. Gama, J. Howe, A. Hülsing, D. Joseph, and D. Yue. The return of the SDitH. In C. Hazay and M. Stam, editors, *EUROCRYPT 2023, Part V*, pages 564–596. Springer, Cham, 2023 (cited on page 1).
- [BBD⁺23] C. Baum, L. Braun, C. Delpech de Saint Guilhem, M. Klooß, E. Orsini, L. Roy, and P. Scholl. Publicly verifiable zero-knowledge and post-quantum signatures from VOLE-in-the-head. In H. Handschuh and A. Lysyanskaya, editors, *CRYPTO 2023, Part V*, pages 581–615. Springer, Cham, 2023 (cited on pages 1–3, 43).
- [BBG⁺24] S. Bettaieb, L. Bidoux, P. Gaborit, and M. Kulkarni. Modelings for generic pok and applications: shorter SD and PKP based signatures. Cryptology ePrint Archive, Report 2024/1668, 2024 (cited on pages 2, 3, 7, 8).
- [BBM⁺24] C. Baum, W. Beullens, S. Mukherjee, E. Orsini, S. Ramacher, C. Rechberger, L. Roy, and P. Scholl. One tree to rule them all: optimizing GGM trees and OWFs for post-quantum signatures. Cryptology ePrint Archive, Report 2024/490, 2024 (cited on pages 9, 19).
- [BJM⁺12] A. Becker, A. Joux, A. May, and A. Meurer. Decoding random binary linear codes in $2^{n/20}$: how $1 + 1 = 0$ improves information set decoding. In D. Pointcheval and T. Johansson, editors, *EUROCRYPT 2012*, pages 520–536. Springer, Berlin, Heidelberg, 2012 (cited on page 41).
- [BM18] L. Both and A. May. Decoding linear codes with high error rate and its impact for LPN security. In T. Lange and R. Steinwandt, editors, *Post-Quantum Cryptography - 9th International Conference, PQCrypto 2018*, pages 25–46. Springer, Cham, 2018 (cited on page 41).
- [CDI05] R. Cramer, I. Damgård, and Y. Ishai. Share conversion, pseudorandom secret-sharing and applications to secure computation. In J. Kilian, editor, *TCC 2005*, pages 342–362. Springer, Berlin, Heidelberg, 2005 (cited on page 4).
- [EB22] A. Esser and E. Bellini. Syndrome decoding estimator. In G. Hanaoka, J. Shikata, and Y. Watanabe, editors, *PKC 2022, Part I*, pages 112–141. Springer, Cham, 2022 (cited on page 41).
- [Fen24] T. Feneuil. The Polynomial-IOP Vision of the Latest MPCitH Frameworks for Signature Schemes. Post-Quantum Algebraic Cryptography - Workshop 2, Institut Henri Poincaré, Paris, France, 2024 (cited on page 3).
- [FJR22] T. Feneuil, A. Joux, and M. Rivain. Syndrome decoding in the head: shorter signatures from zero-knowledge proofs. In Y. Dodis and T. Shrimpton, editors, *CRYPTO 2022, Part II*, pages 541–572. Springer, Cham, 2022 (cited on pages 2, 7, 39).
- [FR22] T. Feneuil and M. Rivain. Threshold linear secret sharing to the rescue of MPC-in-the-head. Cryptology ePrint Archive, Report 2022/1407, 2022 (cited on page 2).
- [FR23a] T. Feneuil and M. Rivain. Threshold computation in the head: improved framework for post-quantum signatures and zero-knowledge arguments. Cryptology ePrint Archive, Report 2023/1573, 2023 (cited on pages 3, 4, 43).

- [FR23b] T. Feneuil and M. Rivain. Threshold linear secret sharing to the rescue of MPC-in-the-head. In J. Guo and R. Steinfeld, editors, *ASIACRYPT 2023, Part I*, pages 441–473. Springer, Singapore, 2023 (cited on pages 1, 3).
- [FS87] A. Fiat and A. Shamir. How to prove yourself: Practical solutions to identification and signature problems. In A. M. Odlyzko, editor, *CRYPTO’86*, pages 186–194. Springer, Berlin, Heidelberg, 1987 (cited on pages 3, 9).
- [IKO⁺07] Y. Ishai, E. Kushilevitz, R. Ostrovsky, and A. Sahai. Zero-knowledge from secure multiparty computation. In D. S. Johnson and U. Feige, editors, *39th ACM STOC*, pages 21–30. ACM Press, 2007 (cited on pages 2, 3).
- [ISN89] M. Ito, A. Saito, and T. Nishizeki. Secret sharing scheme realizing general access structure. *Electronics and Communications in Japan (Part III: Fundamental Electronic Science)*, (9):56–64, 1989. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/ecjc.4430720906> (cited on page 4).
- [MO15] A. May and I. Ozerov. On computing nearest neighbors with applications to decoding of binary linear codes. In E. Oswald and M. Fischlin, editors, *EUROCRYPT 2015, Part I*, pages 203–228. Springer, Berlin, Heidelberg, 2015 (cited on page 41).
- [OTX24] Y. Ouyang, D. Tang, and Y. Xu. Code-based zero-knowledge from VOLE-in-the-head and their applications: simpler, faster, and smaller. In K.-M. Chung and Y. Sasaki, editors, *ASIACRYPT 2024, Part V*, pages 436–470. Springer, Singapore, 2024 (cited on pages 2, 3, 7, 8).
- [ZCD⁺20] G. Zaverucha, M. Chase, D. Derler, S. Goldfeder, C. Orlandi, S. Ramacher, C. Rechberger, D. Slamanig, J. Katz, X. Wang, V. Kolesnikov, and D. Kales. Picnic. Technical report, National Institute of Standards and Technology, 2020. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions> (cited on page 2).